

湖南工商大学本科毕业设计诚信声明

本人郑重声明：所呈交的本科毕业设计视频学习笔记本工具的设计与实现是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议，除文中已经注明引用的内容外，本设计不含任何其他个人或集体已经发表或撰写过的作品成果。对本设计做出重要贡献的个人和集体均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

作者签名：_____

日期：_____年____月____日

摘 要

随着互联网的高速发展，线上教育迎来了快速发展的时机。视频学习作为线上学习的主要媒介，相对于传统学习方式更易使学生视听疲劳，而记笔记可以有效帮助学习者理解巩固知识。目前数字化笔记工具多种多样，但是在市场上却没有专门针对视频的笔记编辑管理工具。本系统基于 B/S 架构和 Vue、Node.js 等技术，实现了一个面向用户的针对视频学习的笔记编辑管理工具。利用 MongoDB 构建文件信息表，实现了网络硬盘，为用户提供了知识管理和整合功能。除此之外，本项目中的资源社区模块为用户提供了知识交流的途径，方便用户进行资源整合，促进知识传播。在经过完整的功能测试后，本系统能满足用户对视频做笔记和知识管理、知识交流的基本需求。本项目在一定程度上弥补了目前视频学习平台在笔记功能上的不足，能够帮助用户提升记笔记的体验。

关键词：Vue；Node；视频学习；知识管理；数字化笔记

ABSTRACT

With the rapid development of the Internet, online education has ushered in an opportunity for rapid development. As the main medium of online learning, video learning is more likely to make students visually and visually fatigued than traditional learning methods, and taking notes can effectively help learners understand and consolidate knowledge. At present, there are various digital note-taking tools, but there is no note-editing and management tool specifically for video in the market. This system is based on the B/S architecture and technologies such as Vue and Node.js, and implements a user oriented note editing and management tool for video learning. By using MongoDB to build a file information table, a network hard drive has been implemented, providing users with knowledge management and integration functions. In addition, the resource community module in this project provides users with a means of knowledge exchange, facilitating resource integration and promoting knowledge dissemination. After undergoing complete functional testing, this system can meet the basic needs of users for video note taking, knowledge management, and knowledge exchange. This project to some extent compensates for the shortcomings of the current video learning platform in note taking functionality, and can help users improve their note taking experience.

KEYWORDS: Vue; Node; video learning; knowledge management; digital notes

目 录

1 绪论	1
1.1 课题背景	1
1.2 国内外研究现状及发展趋势	1
1.3 论文组织结构	2
2 系统需求分析	3
2.1 系统目标	3
2.2 系统功能要求	3
2.3 本章小结	5
3 系统总体设计	6
3.1 软件系统结构设计	6
3.1.1 系统整体架构	6
3.1.2 系统功能结构	7
3.2 系统数据分析	8
3.2.1 E-R 图	8
3.2.2 数据流图	9
3.3 数据库设计	10
3.3.1 用户信息表	11
3.3.2 文件信息表	12
3.3.3 最近查看文件表	12
3.3.4 标签表	13
3.3.5 帖子信息表	13
3.3.6 收藏夹信息表	14
3.4 本章小结	15
4 系统详细设计	16
4.1 登录注册模块设计	16
4.1.1 登录	16
4.1.2 登录状态验证	17
4.1.3 退出登录	19
4.2 个人信息模块	19
4.2.1 基本设置	19

4.2.2 密码修改	21
4.3 视频笔记模块设计	22
4.3.1 打开/切换视频	27
4.3.2 打开/切换笔记	27
4.3.3 笔记编辑与保存	28
4.3.4 视频画面截取	28
4.3.5 视频标记点	30
4.3.6 笔记导出	30
4.4 网盘模块设计	31
4.4.1 文件系统的展示	35
4.4.2 新建/删除文件、文件夹	36
4.4.3 重命名/移动文件、文件夹	38
4.4.4 上传文件	39
4.4.5 下载文件	41
4.4.6 最近查看文件的展示	42
4.5 资源社区模块设计	42
4.5.1 社区帖子的展示与搜索	42
4.5.2 发布帖子	44
4.5.3 管理帖子	45
4.6 我的收藏模块设计	46
4.6.1 收藏帖子	46
4.6.2 管理收藏夹、收藏内容	47
4.7 本章小结	47
5 系统测试与分析	49
5.1 测试概述	49
5.2 功能测试	49
5.2.1 登录注册功能测试	49
5.2.2 视频笔记功能测试	50
5.2.3 网盘功能测试	53
5.2.4 资源社区功能测试	56
5.3 压力测试	59

5.4 测试总结	60
6 总结	61
参考文献	62
致谢	64

视频学习笔记工具的设计与实现

1 绪论

1.1 课题背景

自 2020 年起一场历时三年的疫情席卷全国，受疫情影响各地实行封闭式管理，工厂停工、学校停课^[1]。从高校到中小学，正常的开学及教学工作都受到了不同程度的影响。在“停课不停学”方针的指导下，我国线上教育迎来了一个快速发展的时机，以中国大学 MOOC、学习通为代表的线上学习平台纷纷崛起^[2]。

从媒介类型来看，视频是仍然是线上学习的主要媒介。视频教学能通过视觉和听觉通道传播知识，具备多通道传播的特点；同时视频的制作水准和技术在不断提高，使其呈现方式多样化，例如可以是幻灯片形式、课堂黑板授课、讨论式等多种形式，能够让用户获得更强的临场感。这些特点使其成为线上学习的主要形式。

不同于传统学习方式，视频学习更容易使学生视、听疲劳，学生学习效率下降^{[3][4][5]}。在学习过程中，记笔记是一项重要的学习策略，能够帮助学习者理解和记忆知识，促进新知识与已有知识的整合，且便于学习者在后期对知识进行回顾和巩固。当今信息时代背景下的知识具有数字化和网络特征，借助信息技术手段进行个人知识管理是十分必要的^{[6][7]}。

1.2 国内外研究现状及发展趋势

从笔记软件的发展现状来看，数字化笔记软件是随着计算机技术和网络技术的发展而发展的，其出现具有必然性。数字化笔记软件的发展在云计算技术发展之后也加快了脚步。涌现出许多云笔记类产品，例如微软的 OneNote，网易的有道云笔记，以及近年新推出的云端文档服务，如腾讯文档、谷歌文档等^{[8][9]}。这些工具软件都有以下特性，①笔记数据保存在云端，跨平台共享，用户可以随时随地查看检索笔记。②笔记内容支持多种格式，包括文字、图片、音频等，用户可以灵活组织文档结构^[10]。③进行系统化统一管理，软件可以集中聚合并统一管理所记录的各种信息，便于形成独有、个性化、体系化的知识库^[11]。

但是这些国内外的数字化笔记软件的笔记和视频互动性不强，只能在文档中添加或插入一整段视频，这对于教学类的长视频来说显然是不适用的。哔哩哔哩视频弹幕网站针对视频提供了一款笔记工具，用户可以在笔记内容中为视频添加标记点，以增强

笔记和视频的互动性，让用户在回顾笔记时更快找到视频对应内容。但是这个视频笔记只是哔哩哔哩网站的一个小插件，存在的问题还有很多。①视频笔记依赖于视频平台，当视频被下架或则删除笔记也会失效。②无法离线使用，不支持笔记的导出。③笔记支持的内容较为简单，比如无法在笔记中插入表格。

综上所述，数字化笔记软件的前景广阔，有许多开发人员在研究，但针对视频学习笔记这一细分领域研究相对较少。虽然市场上已经存在像哔哩哔哩视频弹幕网上这样的视频笔记小工具，但是目前还存在较多问题。基于此背景，本课题拟实现一个基于 B/S 架构，支持多种视频来源和形式，支持多种文档格式，能够对视频进行各种记录，支持笔记导出，方便用户进行复习和整理的视频学习笔记工具。

1.3 论文组织结构

论文各章节详细内容如下：

第一章为绪论。阐述了视频学习和笔记工具的现状，列举了市面上多款笔记工具的特点和不足，指出了本课题的背景和现实意义，同时交代了本系统的目的。

第二章为系统需求分析。从用户视角出发为系统设定了功能需求和非功能需求，模拟用户场景分析出系统基本功能需求，并以用例图的形式展示。

第三章为系统总体设计。阐述了本系统使用的系统技术架构和系统功能结构，通过对系统实体和数据流的分析，设计出合适的数据库结构。

第四章为系统详细设计，阐述了系统关键模块关键功能的设计实现与使用的相关技术，其中视频笔记模块和网盘模块是本系统最核心的模块。

第五章为系统测试与分析。利用黑盒测试方法对本系统功能进行了完整测试，绝大部分功能都是能够正常使用的，也发现了一些问题，但都一一解决了。

第六章为总结。阐述了本系统的主要内容和功能，也总结了本系统存在的不足。针对这些不足，对未来可改进的部分进行了展望，以期将系统进一步完善。

2 系统需求分析

2.1 系统目标

本系统作为一款面向用户的 B/S 系统，应达到的目标如下：

- (1) 能够满足用户管理网盘和编辑笔记的基本需求。
- (2) 能够满足用户间的信息交流与资源共享。
- (3) 平台可移植性高，方便移动端使用。
- (4) 系统稳定性好、安全性高。

(5) 数据库安全系数高，对数据库的物理完整性、逻辑完整性、元素完整性、用户鉴别（数据库能够确保每个登录用户被正确识别同时避免非法用户入侵）、可获得性、可审计性等方面提供全面的安全保证。

2.2 系统功能要求

系统共有普通用户和超级管理员两个角色，分别具有不同的功能和权限。经过初步需求分析后，本系统应包含以下功能性需求：

(1) 注册登陆，作为一款可以帮助用户进行知识管理的系统，系统需要通过账号管理来保护用户数据安全。因此用户需要登录系统，获取相关操作的权限后，才能正常使用系统的功能。

(2) 视频播放与解析，用户会常常使用视频播放的功能，本系统应支持视频播放的基本操作，比如拖动进度条、设置视频音量等。同时为了满足用户快速从视频中提取信息的需求，本系统还应支持对视频内容进行解析，比如自动生成字幕、实时翻译等。

(3) 笔记编辑与导出，笔记内容的展示需要支持富文本格式，支持插入表格、图片、代码块等多种样式。为了满足用户的不同场景的不同需求，系统还应支持多种格式的笔记导出。

(5) 文件系统，用户需要对自己的知识库进行构建和管理，这就需要系统实现一个云端文件系统（网盘）功能，并且该文件系统支持上传、移动、删除等多种管理文件的相关操作。

(6) 资源社区，为了增加用户粘性，方便平台用户之间沟通交流，系统需要实现一个资源社区模块。用户可以在该模块发布和浏览帖子，加速知识传播，方便用户进行知识整合。

(8) 消息通知，比如系统需要进行维护升级时，超级管理员可以通过发布通知来告知用户。

(9)管理资源社区内容,超级管理员有权利对资源社区中的内容进行审核与管理,当社区中存在不合法内容时,超级管理员有权利进行删除。

通过对上述功能性需求的进一步分析,得到系统用例图如图 1 所示:

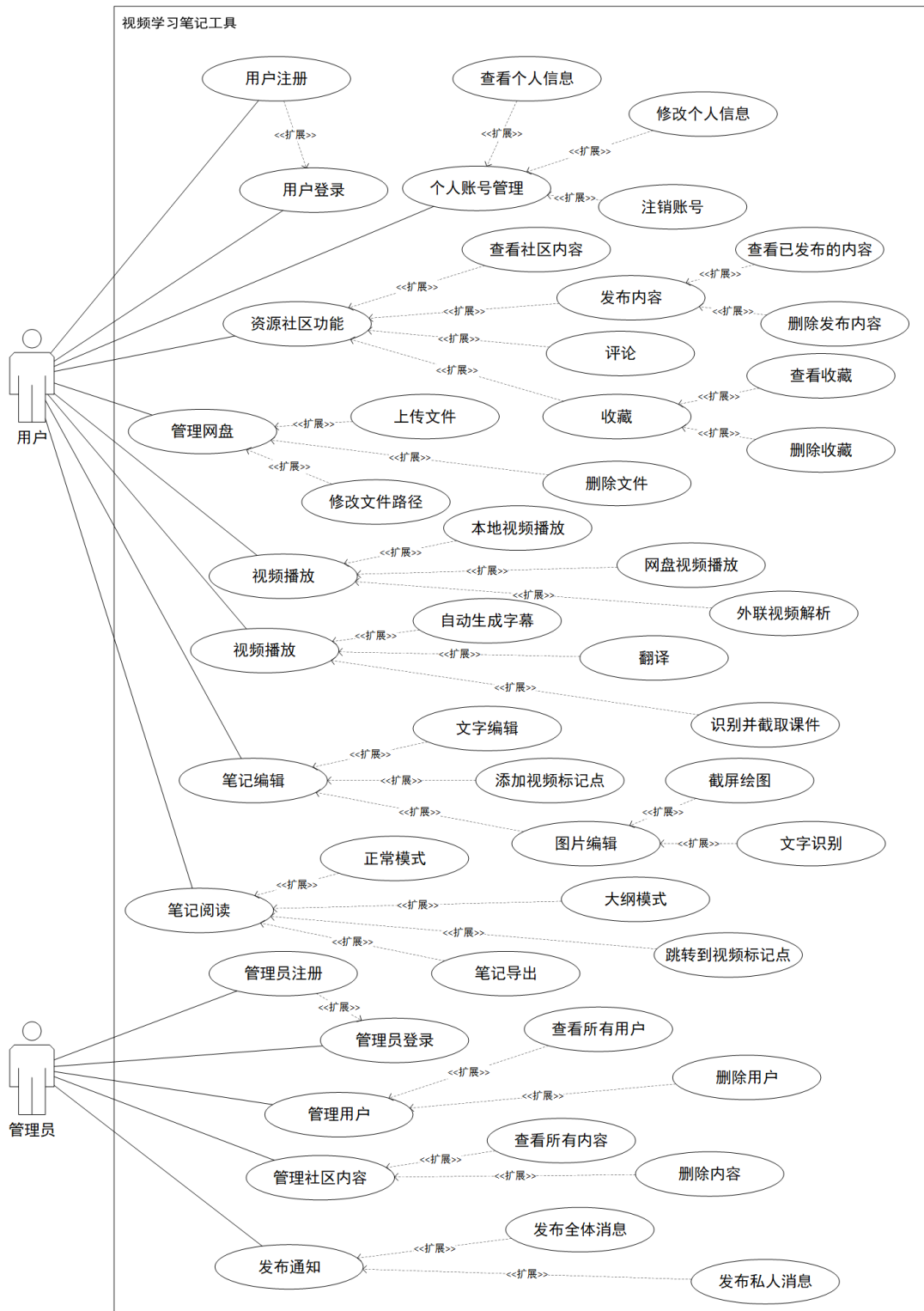


图 1 系统用例图

2.3 本章小结

本章首先对系统目标做出总结，系统目标从用户视角出发，旨在设计一个符合用户习惯、界面美观、数据安全、可移植性强的系统。接着通过对用户场景的模拟，从其中分析出系统的基本需求，主要有注册登录、视频播放与解析、笔记编辑与导出、文件系统、资源社区等需求，其中视频播放与解析、笔记编辑与导出、文件系统是系统的核心需求。后文也是围绕着这几大需求对系统进行设计和实现。

3 系统总体设计

3.1 软件系统结构设计

3.1.1 系统整体架构

系统采用 B/S 架构，在该架构的基础上系统分为 3 层，分别是用户直接接触的表现层、提供系统服务支持的服务层、提供数据支持的数据层^[12]，系统架构图如图 2 所示。

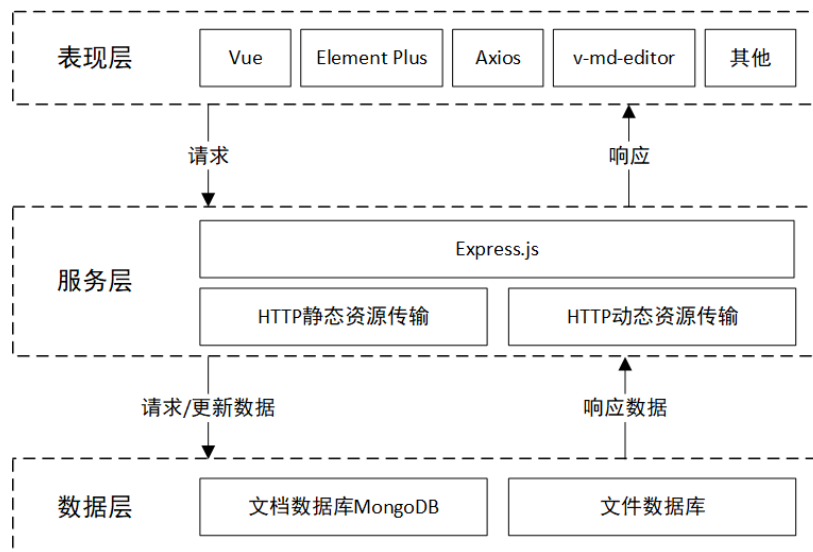


图 2 系统架构图

(1) 表现层，表现层是用户与系统之间的接口，主要负责展示数据和处理用户输入。在 Web 应用程序中，表现层通常由 HTML、CSS 和 JavaScript 等技术实现，它们可让用户和系统进行交互，以实现用户需求^[13]。表现层通过 Ajax 技术与服务层进行 HTTP 通信^[14]，而本系统使用的 Axios 技术是对 Ajax 的封装。

(2) 服务层，服务层是系统的核心，它提供了各种服务和业务逻辑。服务层负责处理表现层传递过来的请求，并调用数据层的相关方法，从而实现对数据的增删改查等操作。本系统的服务层由 Node.js+Express.js 实现，Node.js + Express.js 搭建服务器有以下优点，轻量级、高效性能、跨平台运行、强大的可扩展性^{[15][16]}。

(3) 数据层，数据层是系统的数据存储和访问层，它负责管理数据库、文件系统等数据存储设备，并提供相应的数据访问接口。本系统的数据层由 MongoDB 实现，以保证对数据的有效管理和存储^[17]。

3.1.2 系统功能结构

系统功能包括：

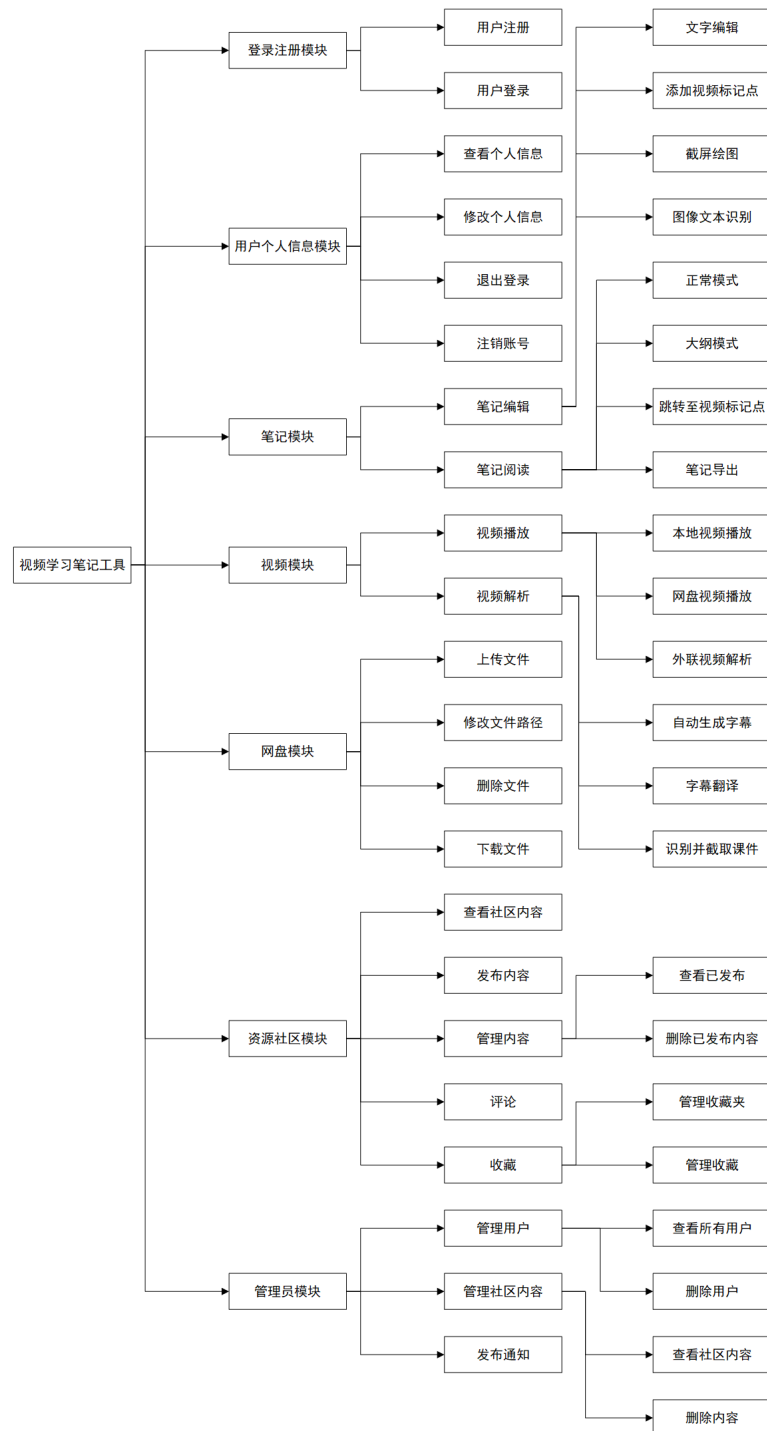


图 3 系统功能结构图

(1) 注册登陆：用户登录与注册。

- (2) 用户个人信息模块：查看个人信息、修改个人信息、退出登录、注销账号
- (3) 笔记模块：笔记编辑（文字编辑、添加视频标记点、截屏绘图、文字识别）、笔记阅读（正常模式、大纲模式、跳转到视频标记点）。
- (4) 视频模块：视频播放（本地视频播放、网盘视频播放、外链视频解析）、视频解析（自动生成字幕、翻译、识别并截取课件）
- (5) 资源社区模块：查看社区内容、评论、收藏、发布内容、管理内容（查看已发布、删除已发布内容）。
- (6) 网盘模块：上传文件、删除文件、修改文件路径。
- (7) 管理员功能：管理用户、管理社区内容、发布通知。

系统功能结构图如图 3 所示。

3.2 系统数据分析

3.2.1 E-R 图

经过分析得到系统存在六个实体，分别是用户、文件、最近查看文件、标签、帖子、收藏夹，实体的属性以及实体之间的关系如图 4 所示。

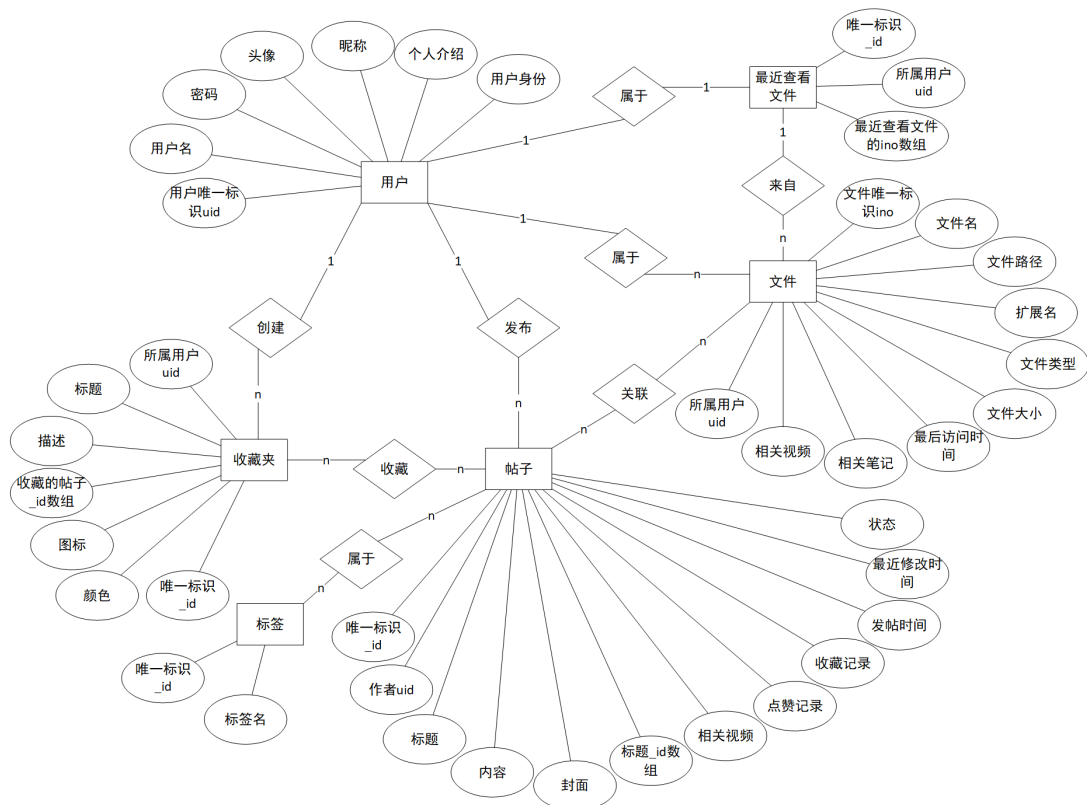


图 4 系统整体 E-R 图

用户实体包含用户的所有属性，比如用户唯一标识 `uid`、用户名等；文件实体包含文件在系统中需要使用到的信息，比如文件唯一标识 `ino`、文件名、文件路径等。每个文件只能属于一个用户，而一个用户可以拥有多个文件，所以用户和文件属于一对多的关系；最近查看文件实体，包含唯一标识 `_id`、所属用户 `uid`、最近查看文件的 `ino` 数组三个属性。每个用户只有一个最近查看文件列表，一个最近查看文件列表只属于一个用户。一个文件只能存在于一个最近查看文件列表中，一个最近查看文件列表中包含多个文件。所以用户和最近查看文件是一对一的关系，最近查看文件和文件是一对多的关系；

标签实体只包含标签唯一标识 `_id` 和标签名两个属性；帖子实体，包含帖子所需要的所有属性，比如作者 `uid`、标题、内容等。一个用户可以发布多个帖子，而一个帖子只能被一个用户发布，所以用户和帖子直接存在一对多的关系。一个帖子可以拥有多个标签，而一个标签可以被多个帖子选择，所以帖子和标签之间是多对多的关系。一个帖子可以关联多个文件，一个文件也可以被多个帖子关联，所以帖子和文件直接存在多对多的关系；收藏夹实体用来表示收藏夹的所有信息，包括所属用户 `uid`、收藏的帖子数组等，一个用户可以创建多个收藏夹，而一个收藏夹只能属于一个用户，所有用户和收藏夹之间的关系是一对多的关系。一个收藏夹中可以收藏多个帖子，而一个帖子也可以被多个收藏夹收藏，帖子和收藏夹之间的关系属于多对多的关系。

3.2.2 数据流图

本系统的顶层数据流图如图 5 所示，系统只跟普通用户和管理员进行数据交互。将系统内部进行具体描述，得到 0 层数据流图，如图 6 所示。

(1) 登录注册模块，用户和管理员都需要输入账号密码才能进行登录和注册功能，账号密码被存储在用户表中。

(2) 个人模块，用户可以输入用户信息对其存储在数据库中的进行修改。

(3) 视频笔记模块，用户可以根据文件数据和文件信息播放视频和查看笔记；当用户编辑笔记时，输入的数据会被存入到文件和文件表中。

(4) 网盘模块，文件数据、文件信息和最近查看文件 `ino` 通过网盘模块的处理可以在前端展示出文件系统和最近查看文件信息；用户在上传文件等文件系统操作时，其数据也会被写入到文件、文件表和最近查看文件表中。

(5) 资源社区模块，标签信息、收藏夹信息、帖子信息和文件数据通过资源社区模块的处理后，可以在前端展示资源社区的完整内容；而用户发布帖子、管理收藏夹时产

生的数据也会被存入标签表、帖子表和收藏夹表中。

(6) 管理员模块，管理员可以根据用户列表和帖子列表进行浏览；管理员对用户和帖子进行管理时产生的数据会被重新写入用户表和帖子中；管理员还可以通过管理员模块对用户发布通知，用户会收到通知信息。

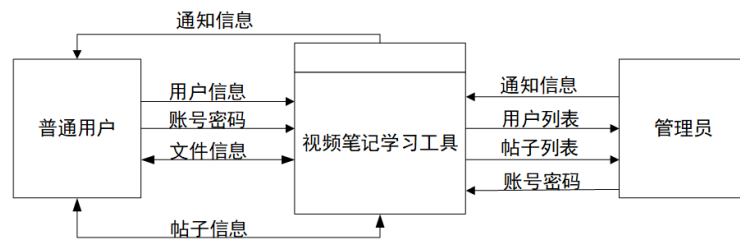


图 5 系统顶层数据流图

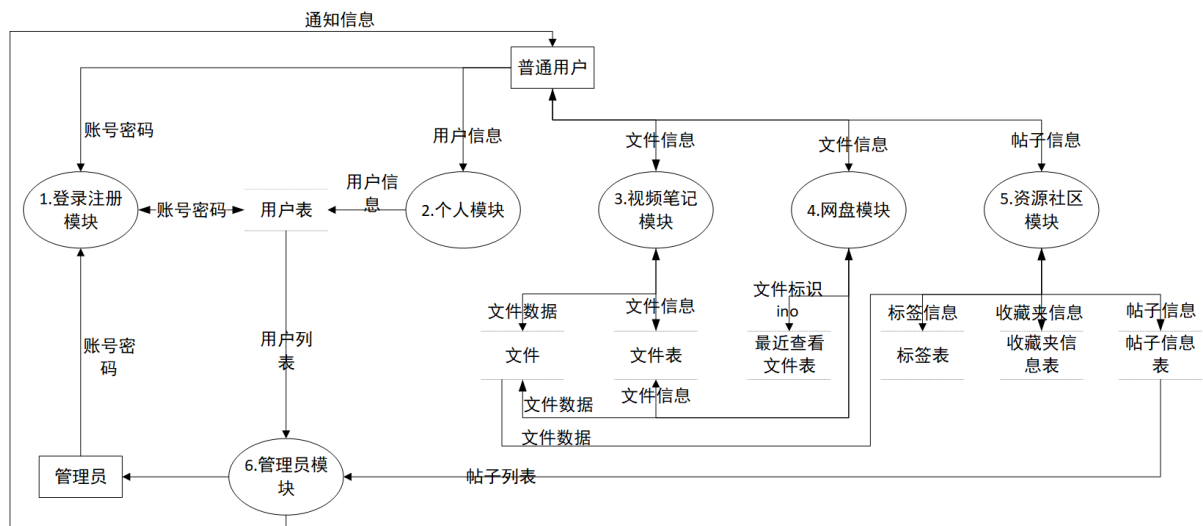


图 6 系统 0 层数据流图

3.3 数据库设计

本系统选用的数据库是文档型数据库 MongoDB，与传统关系数据库相比，非关系数据库在 Web 应用中拥有更好的扩展性和更优越的性能。随着互联网技术的不断发展，要存储的数据越来越复杂多样。由于关系数据库固定的表结构存储模式和有限的字段类型，无法适应一些非结构化数据的存储。而非关系型数据库则主要针对大数据环境下的数据特点对其诸多功能中原有的限制进行扩展和突破，更好地适应了信息时代的发展要求^[18]。MongoDB 文档型数据库具备分布式数据存储支持能力，MongoDB 具有高性能、低成本的横向扩展能力、部署方便等优势^[19]。

根据 3.2.1 E-R 图设计数据库表如下所述。其中，有两个关键的表，用户信息表和文件信息表。作为一款 C 端产品，用户的信息必不可少，用户的所有信息都存放在用户信息表中，其中用户 uid 为用户身份的唯一标识，它也作为将作为连接字段存储在其他表中；本系统绝大部分功能性操作都与存储在服务器上的文件有关。将文件的信息存储在一张表中，根据其唯一标识很容易就可以获取一些信息，比如文件存放的位置，文件的大小状态等。

3.3.1 用户信息表

本系统是面向个人用户使用的，需要对用户信息进行规范化管理。用户的基本信息包括用户唯一标识 uid、账号、密码、用户昵称、个人介绍、头像。基于系统安全性，防止密码被意外传递到客户端，本系统将用户的密码单独存储在一张表中，如表 1 所示。在进行身份验证时，只需要检索用户密码表即可。

用户信息表中则存放除密码外的其他用户信息，其中，uid 来自用户创建账户时用户密码表自动生成的 uid；role 字段表示用户身份，当值为 user 时，即为普通用户，当值为 admin 时，即为系统管理员；综上所述，可得用户信息表如表 2 所示。

表 1 用户密码表

字段名称	数据类型	是否可为空	字段描述
uid	objectId	否	用户唯一标识
username	string	否	用户名
password	string	否	密码

表 2 用户信息表

字段名称	数据类型	是否可为空	字段描述
uid	string	否	用户唯一标识
username	string	否	用户名
avatar	string	是	用户头像
nickname	string	是	用户昵称
introduce	string	是	用户个人介绍
role	string	否	用户身份（user admin）

3.3.2 文件信息表

本系统允许用户创建云端文件以及上传文件，对所有存储在服务器的文件，其基本信息都会被存储到数据库当中。文件信息表如表 3 所示。

在操作系统中，每个文件都有一个 `inode` 号码，操作系统使用 `inode` 号码来识别不同的文件，当文件进行重命名或者移动操作时，其 `inode` 号码也不会发生变化。利用上述特点，本系统使用文件的 `inode` 号码作为文件的唯一标识，在数据库上即为 `ino` 字段；在文件被创建或者上传后，系统会将文件的类型存储到 `type` 字段中，根据 `type` 值的不同，文件拥有的功能也不同。比如当 `type` 为 `video` 时，文件信息中会在 `connectNotes` 字段中存储与其相关联的笔记文件的 `ino` 数组；

表 3 文件信息表

字段名称	数据类型	是否可为空	字段描述
<code>ino</code>	<code>string</code>	否	文件唯一标识
<code>name</code>	<code>string</code>	否	文件名
<code>path</code>	<code>string</code>	否	文件在服务端的绝对路径
<code>ext</code>	<code>string</code>	否	文件扩展名
<code>type</code>	<code>string</code>	否	文件类型（ <code>video</code> <code>note</code> <code>image</code> <code>other</code> ）
<code>size</code>	<code>string</code>	否	文件实际大小
<code>atime</code>	<code>date</code>	否	文件最后一次被访问时间
<code>uid</code>	<code>string</code>	否	文件所属用户的 <code>uid</code>
<code>connectNotes</code>	<code>string[]</code>	是	<code>video</code> 链接的笔记 <code>ino</code> 数组，仅当 <code>type</code> 为 <code>video</code> 时有该字段内容
<code>connectVideo</code>	<code>string</code>	是	<code>note</code> 链接的视频 <code>ino</code> ，仅当 <code>type</code> 为 <code>note</code> 时，有该字段的内容

3.3.3 最近查看文件表

为了方便用户快速打开最近查看或修改的文件，本系统利用最近查看文件表，来记录用户最近使用过的文件，如表 4 所示。其中 `files` 字段是一个大小为 20 的队列类型，当用户查看或打开某个文件时，这个文件的 `ino` 会被压入队列，`uid` 字段则标识了用户所

属。

表 4 最近查看文件表

字段名称	数据类型	是否可为空	字段描述
_id	objectId	否	唯一标识
uid	String	否	用户标识
files	string[]	否	最近查看的文件的 ino 数组，最多只保存 20 条数据

3.3.4 标签表

为了实现将资源社区的信息精准推送给用户，本系统要求每个发布的帖子都至少需要有一个标签。所有可选标签都来自数据库，用户也可以自信新增标签。在数据库中标签表仅有两个字段，一个是唯一标识 `_id`，一个是标签名 `name`，如表 5 所示。Mongodb 文档型数据库为系统扩展提供了许多可能，这使标签表以后可以很轻松地新增字段。

表 5 标签表

字段名称	数据类型	是否可为空	字段描述
_id	objectId	否	唯一标识
name	string	否	标签名

3.3.5 帖子信息表

本系统的资源社区模块是依赖用户发表的帖子来进行运转的，因此需要对帖子信息进行规范化、合理化地存储管理。基本信息包括帖子标识、帖子作者、帖子标题、帖子内容、帖子封面、帖子标签、发布的视频信息、点赞记录、收藏记录、发帖时间、最近修改时间、帖子状态，如表 6 所示。

其中帖子唯一标识 `_id` 由 mongodb 数据库自动生成；帖子内容 `content` 字段记录的是一个文本文件的 `ino` 值，这样做的好处是可以随写随存，防止由于帖子内容过多而导致从数据库存取信息到数据库效率低下；`tags` 字段、`videoList` 字段、`like` 字段和 `collect` 字段都是字符串数组类型，`tags` 字段记录当前帖子的标签 `_id`，`videoList` 字段记录与帖子

相关的视频文件的 **ino**，**like** 和 **collect** 字段记录的都是用户 **uid**；**status** 字段表示当前帖子的状态，目前存在两种状态，当值为-1 时，表示帖子为草稿状态，当值为 1 时，表示帖子为已发布状态，可以正常查看；

表 6 帖子信息表

字段名称	数据类型	是否可为空	字段描述
_id	string	否	帖子唯一标识
uid	string	否	帖子作者 uid
title	string	是	帖子标题
content	string	否	帖子内容文件 ino
cover	string	是	帖子封面图片 ino
tages	string[]	否	帖子标签 ino 数组
videoList	string[]	是	发布视频 ino 数组
like	string[]	否	点赞记录
collect	string[]	否	收藏记录
ctime	date	是	发帖时间
mtime	date	是	最近修改时间
status	number	是	帖子状态（值为-1，表示草稿状态，值为 1，表示已发布状态）

3.3.6 收藏夹信息表

收藏功能作为资源社区模块的一大核心功能，表现在数据库上其实并不难实现。收藏夹基本信息包括，唯一标识、所属用户、收藏夹标题、收藏夹描述、收藏的帖子，收藏夹图标、收藏夹标签颜色，如表 7 所示。其中 **list** 字段的类型为字符串数组，用来记录收藏夹收藏的帖子，使用帖子的唯一标识 **_id** 来记录。为了提升用户的使用感，本系统为收藏夹信息表添加了 **icon** 和 **color** 字段，分别用来设置收藏夹标签在前端展示的图标和背景颜色。

表 7 收藏夹信息表

字段名称	数据类型	是否可为空	字段描述
_id	objectId	否	唯一标识
uid	string	否	所属用户
title	string	否	收藏夹标题
des	string	是	收藏夹描述
list	string[]	否	收藏的帖子
icon	string	否	收藏夹图标
color	string	否	收藏夹标签颜色

3.4 本章小结

本章为系统的总体设计，首先介绍了软件系统的整体架构和系统的功能结构，系统基于 B/S 架构实现，系统功能共分为七大模块。通过 E-R 图和数据流图，得到本系统所有的实体、实体之间的关系和数据流走向，本系统一共存在六个实体。最后设计出适用于 MongoDB 数据库的表结构。

4 系统详细设计

4.1 登录注册模块设计

4.1.1 登录

本系统需要用户登录验证成功后方可正常使用，用户没有账号的情况下需要先进行注册。登录界面和注册界面，如图 7 所示。登录注册的流程图如图 8 所示。

（1）设计思路

登录：用户在页面输出正确的账号和密码并提交后，服务器会调用数据库进行验证，当账号或密码不正确时，会在页面向用户展示错误信息；

注册：用户在登录页面点击注册按钮后，会显示注册页面。要求用户输入的账号唯一，不能和其他用户的账号相同。



图 7 登录注册界面图

（2）相关技术

MD5：随着信息技术的不断发展，信息泄露、信息安全等问题也接踵而来。将网站用户密码通过加密存储可以降低网站的风险性，防止用户信息泄露。实现加密存储的方法技术有多种，其中 MD5 信息摘要算法在加解密技术上被广泛使用。它是一种密码散列函数，可以产出一个 128 的散列值，用于确保信息一致。MD5 算法具有压缩性、容易计算、抗修改性、强抗碰撞等特点^[20]。基于这些特点，MD5 在进行用户密码的加解密时，首先把用户密码进 MD5 加密后存入数据库，当用户登录时，系统把用户输入的密码计算成 MD5 值，然后再去和保存在数据库中的 MD5 值进行比较，进而确定输入的密码是否正确。通过这样的步骤，系统可以在并不知道用户密码的明码情况下验证用户登录系统的合法性^[21]。

用户密码与用户信息分离：除了用户进行身份验证的时候，绝大多数情况下，客户端只想获得用户的基本信息。实现这个需求有两种方法，一是在从数据库获得数据时将用户密码剔除，二是将用户密码单独存放在一张表中。本系统采用的是方法二，用户密码与用户信息分离，将用户密码单独存放在一张表中，这样做可以防止用户密码意外地泄露到客户端，保障用户信息安全。

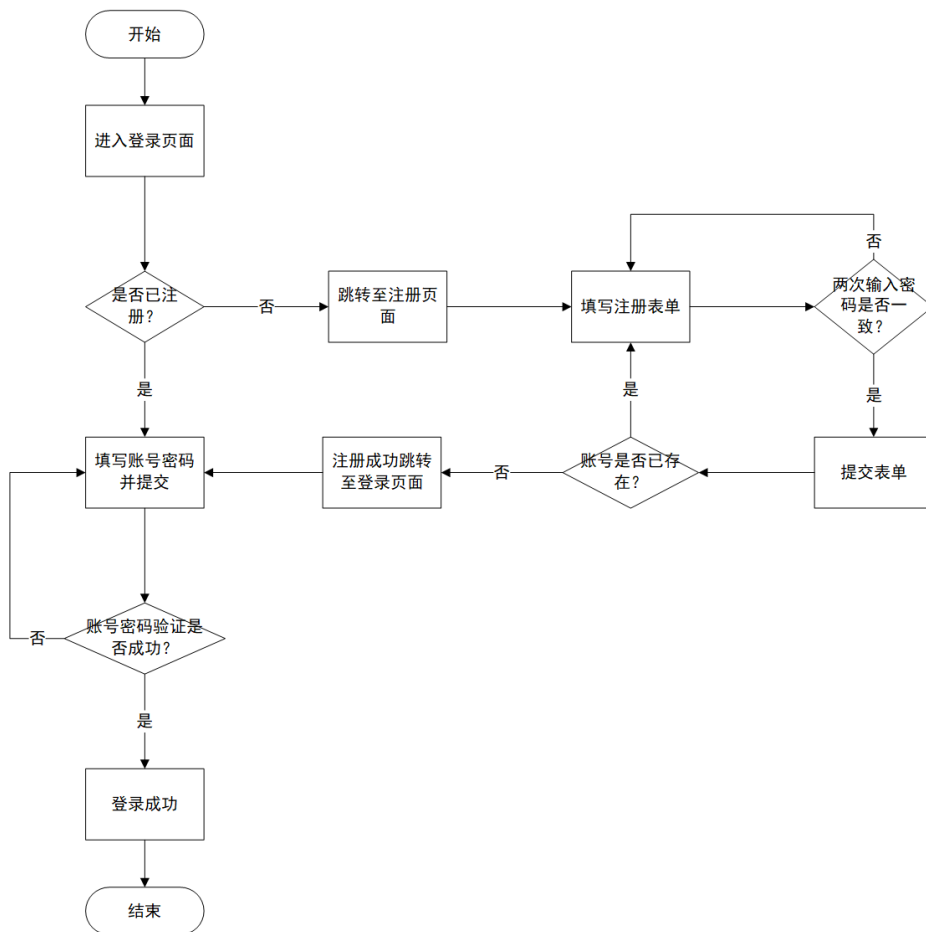


图 8 登录注册流程图

4.1.2 登录状态验证

(1) 设计思路

在验证用户身份合法后，服务器会返回 token 字符串作为用户的登录凭证。token 字符串由用户重要信息加上指定的密钥进行非对称加密算法得到，同时 token 字符串具有一定的时效性，一定时间后 token 字符串会主动失效。客户端获得 token 字符串后，需要将其保存，在以后的每次请求中，都需要将 token 字符串写入请求头的 Authorization

字段中。服务器对需要确认用户身份的请求，都会验证 token 字符串的合法性，当请求头中没有 Authorization 字段，或者 token 不合法时，服务器会拒绝处理请求，并抛出类型为 UnauthorizedError 的错误。客户端收到 UnauthorizedError 的响应时，则会跳转至登录页面要求重新验证用户身份，登录状态验证流程如图 9 所示。

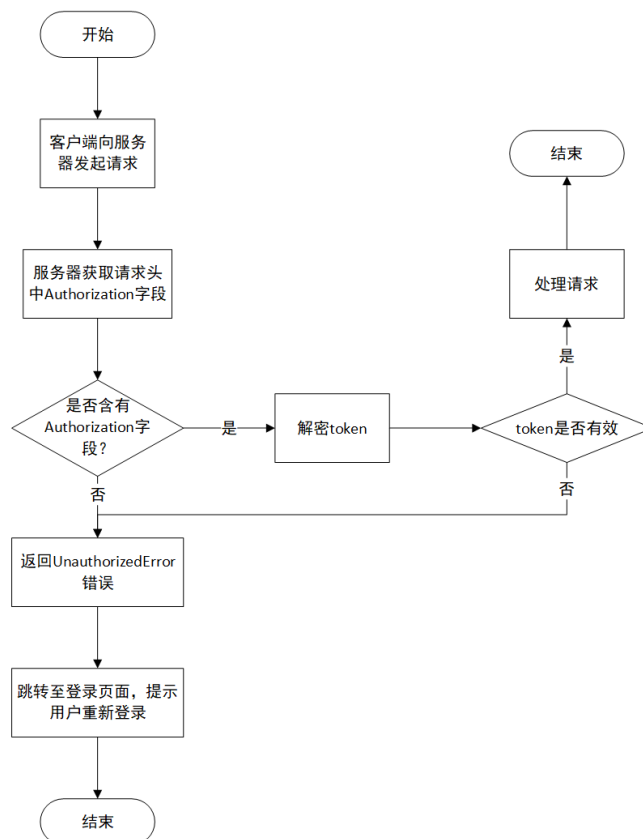


图 9 登录状态验证流程图

服务器端核心代码：

```
const tokenStr = jwt.sign({ username: userInfo.username, uid }, secretKey, { expiresIn: '2h' }); // 将用户名和用户 uid 进行加密得到 token 字符串，设置有效时间为 2h
```

```
app.use(eJWT.expressjwt({ secret: secretKey, algorithms: ['HS256'] })).unless({ path: [/^\/api\/$/, /^\/test\/$/, /^\/media\/$/]})); // 对每次请求服务器都执行该中间件，验证 token 的合法性
```

客户端核心代码：

```
localStorage.setItem("token", res.token); // 登录成功后将 token 保存在 localStorage 中
```



```
config.headers['Authorization'] = `Bearer ${ localStorage.getItem('token')}`; // 为每一次请求的请求头的 Authorization 写入 token 字符串
```

（2）相关技术

JWT: 随着 Web 应用技术的不断发展, 前后端分离的开发模式已成为了主流。基于 Session 和 cookie 的传统 Web 程序认证的弊端逐步显现出来。JWT 是一种基于 JSON 格式, 用于 Web 应用环境下各方之间传递声明信息的开放标准^[22]。它与 Session 机制最大的区别就是一个将用户认证信息保存在服务端, 一个保存在客户端, 大大减轻了服务器端的存储压力。

localStorage: 对于服务器从得到 token 字符串, 客户端需要将其保存以便下次请求时使用, 这就涉及到浏览器的本地存储。浏览器本地存储的常用方式有三种, 分别是 Cookie^[23]、localStorage 和 sessionStorage。其中 sessionStorage 中保存的数据仅在当前会话中有效, 这意味着当用户关闭浏览器或当前窗口时, 下次打开需要用户重新登录, 这显然是不合理的要求。Cookie 和 localStorage 中保存的数据都可以永久有效, 但是 Cookie 中保存的数据是可以直接通过浏览器看到的, 因此本系统使用 localStorage 来保存 token^[24]。同时对于一些全局的数据, 比如用户基本信息, 也是通过 localStorage 来保存的。

4.1.3 退出登录

在用户登录之后, 避免不了用户想退出登录, 然后使用其他账号登入系统的需求。基于 4.1.1 和 4.1.2 可知, 退出登录只需要客户端将用于前后端验证的 token 字符串缓存清理掉, 然后跳转到登录界面即可。

核心代码:

```
sessionStorage.clear(); // 清理 sessionStorage 中数据
localStorage.clear();   // 清理 localStorage 中数据, 包括 token
this.$router.replace({ path: '/login' }); // 重定向至登录界面
```

4.2 个人信息模块

4.2.1 基本设置

（1）设计思路

个人信息模块的基本设置包括基本信息的展示与修改, 展示的信息有用户 uid、用户名、昵称、个人介绍和头像, 其中昵称、个人介绍和头像是可修改的, 如图 10 所示。

在输入框填写新数据后，点击更新按钮即可修改成功。

对于头像的更换则需要先将头像图片上传到服务器中，上传成功后获得图片在服务器上的 ino 值，根据 ino 值为图片文件在数据库的 file 中新建一行，之后再将用户在数据库中的 avatar 字段改为 ino 值，最后将 ino 值返回给客户端。客户端就可以根据 ino 值展示出最新的图片。上传图片的流程如图 11 所示。

服务器向客户端发送图片数据核心代码：

```
const fileReader = fse.createReadStream(filePath); // 创建读取文件流
fileReader.pipe(res); // 使用管道流将文件压入 HTTP 相应流中
```

个人信息

uid 63ec4a67fce25f2510d5fbd0

用户名 admin

昵称

个人介绍

更新个人信息

修改密码



更换头像

图 10 个人信息界面

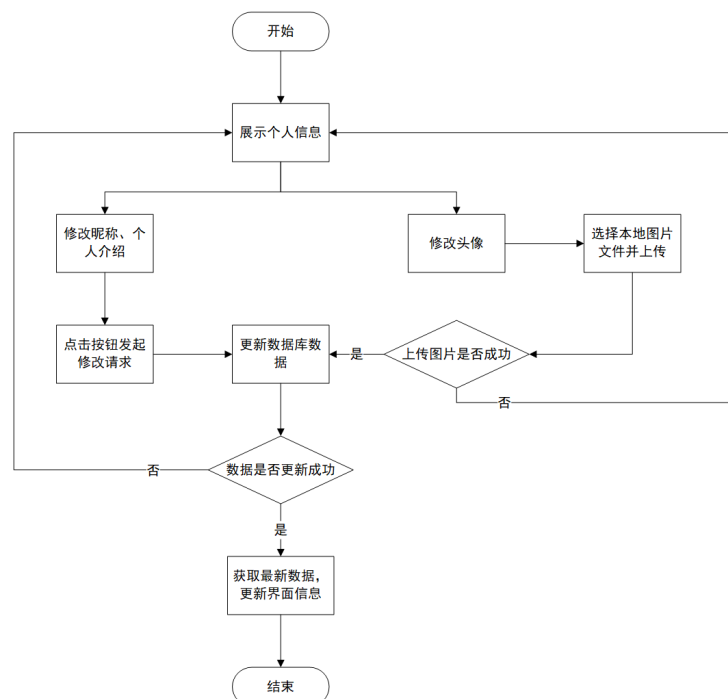


图 11 上传图片流程图

(2) 相关技术

fs-extra: fs 模块是 Node.js 官方提供的、用来操作文件的模块。它提供了一系列的方法和属性，用来满足用户对文件的操作需求，该模块的所有方法都有同步和异步两种方式。而 fs-extra 是 fs 的扩展，添加了 fs 模块中不包含的文件系统的方法。在更换头像的需求中，本系统使用 fs-extra 模块实现了将新建文件夹、buffer 数据保存为本地文件、合并文件、读取文件流等操作。

4.2.2 密码修改

图 12 修改密码界面图

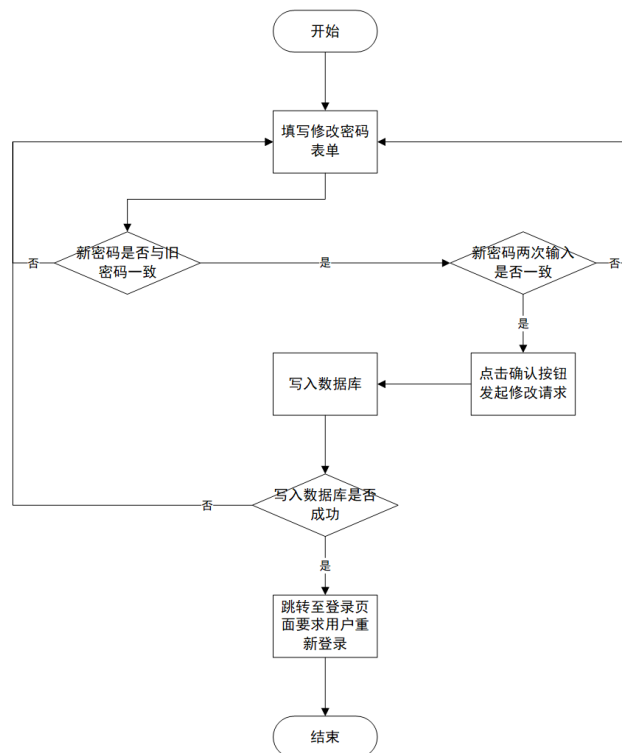


图 13 修改密码流程图

（1）设计思路

个人信息模块的另一功能是对密码的修改设置，当在个人信息的基本设置页面点击修改密码按钮时，会显示密码修改框，如图 12 所示。为提高账号安全性，在修改密码时需要先输入旧密码，后端验证旧密码成功后才能修改密码成功。用户在设置新密码时，需要输入两次，以确认新密码无误。当输入不合法时会表单会提示相应错误，如图所示。修改密码的流程图如图 13 所示。

4.3 视频笔记模块设计

视频笔记模块是本系统最重要的模块，也是用户会使用得最多的模块，决定了用户对本系统最终评价。本模块又分为两个模块，视频模块和笔记模块，视频模块主要负责视频的播放和处理，笔记模块主要负责笔记的编辑与预览。同时这两个模块之间的交互对用户体验也非常重要。下面将从打开视频、新建笔记到保存笔记的所有步骤来演示视频笔记模块，演示步骤如图 14 所示。

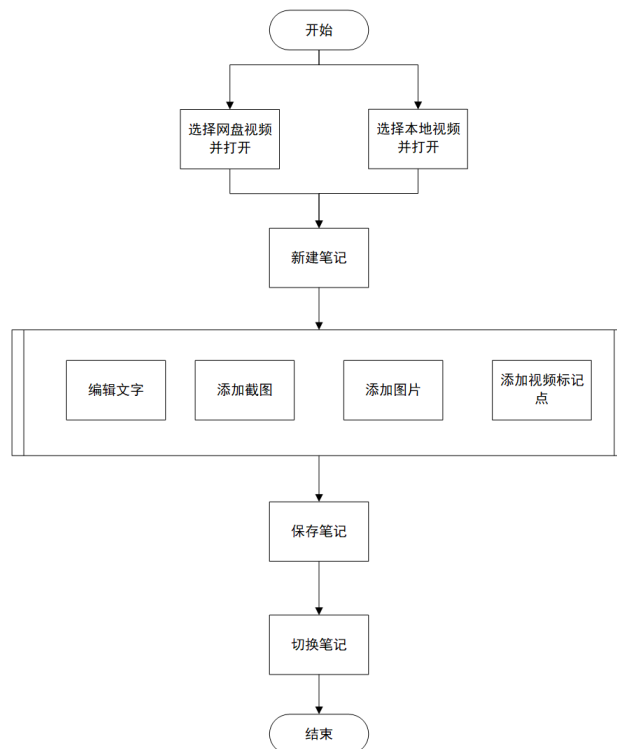


图 14 视频笔记模块演示步骤

（1）播放视频，用户在网盘页面点击网盘中的视频文件，打开网盘视频；或者点击播放本地视频按钮，选择本地视频进行播放。如图 15 所示；



图 15 播放视频操作界面

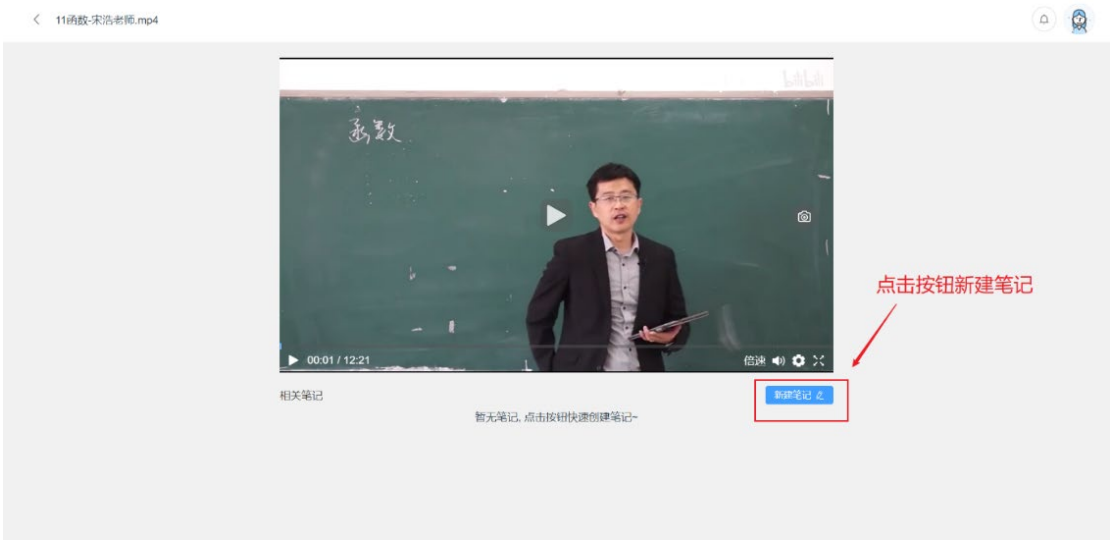


图 16 视频播放界面图

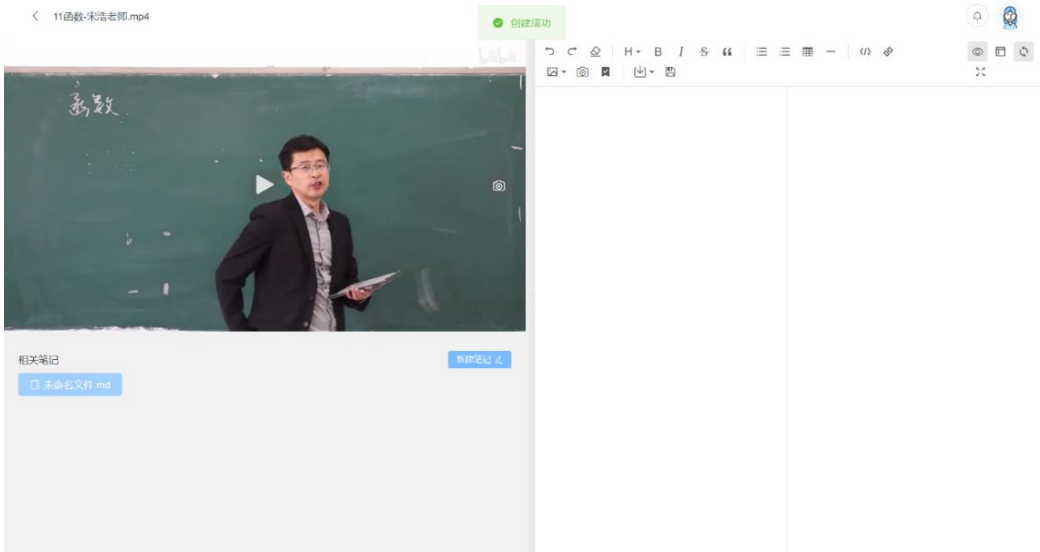


图 17 创建笔记成功界面图

(2) 新建笔记，打开视频后界面如图 16，点击视频播放器下方的新建笔记按钮，会为该视频新建一个相关的笔记，如图 17 所示。

(3) 编辑笔记，打开笔记后，用户可以在右侧编辑器中对笔记进行编辑，编辑器上方按钮可以实现对 markdown 文本的快捷操作，如图 18 所示。



图 18 笔记编辑器界面说明图

(4) 添加图片，用户可以为笔记添加图片，视频截图和上传本地图片两种方式，如图 19 所示。其中点击视频截屏，会截取视频当前帧画面，并添加到笔记内容中。



图 19 添加图片演示图

(5) 添加视频标记点，用户点击按钮，会在笔记中添加视频当前时间点，如图 20 所示。在笔记预览中点击视频标记点，可以使视频跳转到指定位置。

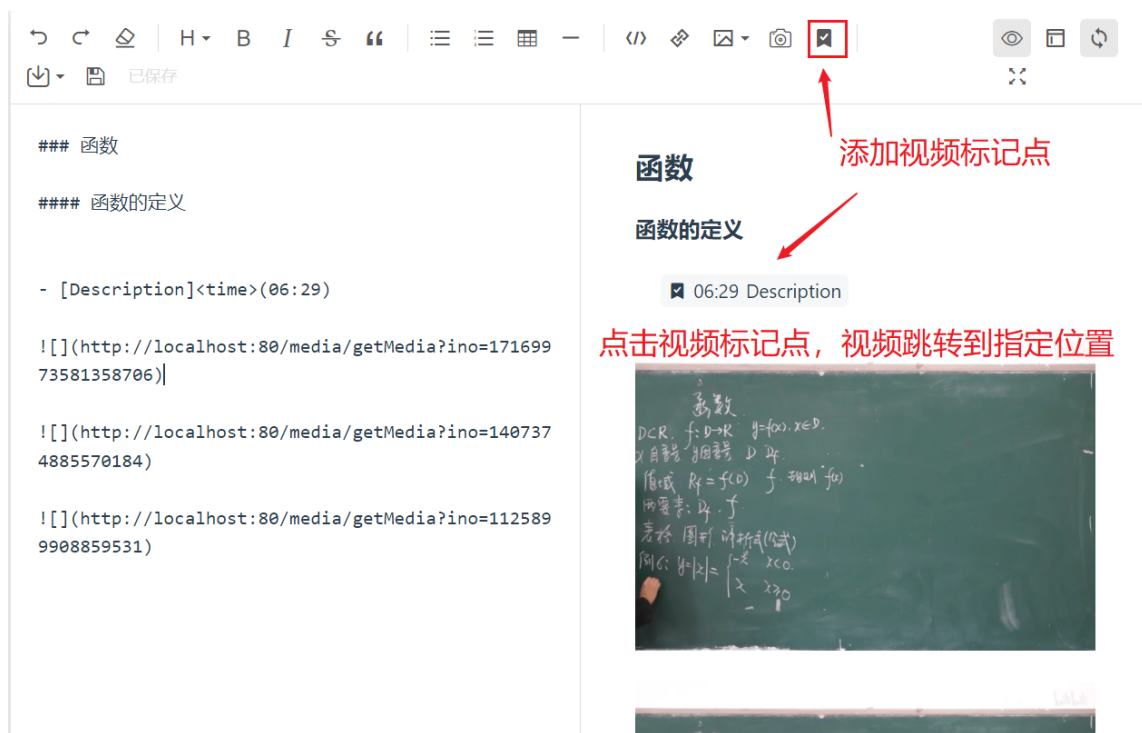


图 20 添加视频标记点演示图

(6) 笔记导出，目前支持笔记导出为 md 格式和 PDF 格式，如图 21 所示。点击按钮，浏览器会自动下载文件。



图 21 导出笔记操作界面说明图

(7) 保存笔记，当笔记内容发生更改，系统会在 10s 后自动保存笔记，用户也可以点击保存按钮，手动进行保存，如图 22 所示。



图 22 保存笔记操作演示图

(8) 切换笔记，当视频关联的笔记超过 1 个时，可以点击播放器下方笔记列表，切换笔记，如图 23 所示。



图 23 切换笔记演示图

4.3.1 打开/切换视频

当点击文件系统中的视频时，会跳转到视频播放页面，如图 16 所示。该界面包含一个基本的视频播放器，支持播放、暂停、滑动条跳转播放、音量调节、播放倍速设置、全屏等视频操作。当存在相关的视频或笔记时，会在播放器下面展示出来。点击视频列表中的视频即可进行视频的切换。

本系统还支持本地视频的播放，在最近播放界面或者我的收藏页面点击播放本地视频的菜单项，会弹出窗口要求用户选择本地可播放视频，选择本地视频后也会跳转到图界面。

关键代码：

```
let binaryData = []; // 新建二进制数组
binaryData.push(file); // 将 File 对象压入数组
this.src = window.webkitURL.createObjectURL(new Blob(binaryData)); // 生成 URL
```

4.3.2 打开/切换笔记

（1）实现思路

用户有两种方式打开笔记，一是点击文件系统中的笔记文件，二是点击文件系统中视频文件中显示的相关笔记中的笔记。前者仅会打卡笔记；后者会将笔记和视频一同打开。当用户打开某个笔记时，服务器会读取整个笔记文件，并将其发送给客户端，客户端接收到的数据为 Blob 流数据，需要将其转化为字符串才能在页面中展示。

当播放的视频存在其他相关视频时，用户点击其他视频，可以实现笔记的切换。目前系统只支持对网盘中笔记文件的展示与编辑，不支持本地文件。

关键代码：

```
let reader = new FileReader(); // 创建一个文件读取示例
reader.readAsText(res, "utf-8"); // 将接收到的 Blob 对象以 utf-8 编码转换为字符串，
此方法为异步方法
```

```
reader.onload = (e) => { this.content = reader.result; }; // 转换完成后，将文件内容保存
```

（2）相关技术

FileReader：该对象允许 Web 应用程序异步读取存储在用户计算机上的文件（或原始数据缓冲区）的内容，使用 File 或 Blob 对象指定要读取的文件或数据。该接口提供 readAsArrayBuffer()、readAsBinaryString()、readAsDataURL()、readAsText()方法，可以

分别将文件或数据转化为 `ArrayBuffer` 类型、二进制格式、base64 格式的 URL 以及字符串类型。

4.3.3 笔记编辑与保存

本系统对笔记内容采用 `markdown` 的语法格式，可以使普通文本具有一定格式，支持标题、粗体、斜体、删除线、引用、代码块、表格、序号、链接、分割线、上传图片等功能。用户打开笔记文件即可进行编辑。

对于网页端编辑类应用，可能常常出现因为网络或其他问题与服务器断连，从而导致数据丢失、同步错误、不稳定性、信息泄露、用户体验下降等问题。为了解决这个问题，本系统对笔记内容进行实时保存，当系统检测到笔记内容发生更改后，会打开一个定时器，10 秒后对文本内容进行一次保存。

关键代码：

```
if(this.isChange) { // 当文件内容发生更改时
    clearTimeout(this.timeout); // 清除之前的定时器
    this.timeout = setTimeout(()=>{this.onSave();},10 * 1000); // 10 秒后进行一次保存
}
```

4.3.4 视频画面截取

(1) 实现思路

在编辑笔记的过程中，用户常常有对视频画面进行截取并保存的需求。本系统为用户提供了两种视频画面截取的方法。

一种是在笔记编辑器上方菜单的截屏按钮，如图 19 所示。点击按钮，系统会直接获取视频当前帧的画面，并将其转换成 `base64` 格式。客户端将图片数据上传到服务器后，服务器把 `base64` 数据转换为 `Buffer` 类型，创建文件并写入数据。上传文件成功后，会返回一个 `ino` 值给到客户端，客户端在笔记内容后面新增图片的地址，即可在笔记预览界面中展示已上传的图片。

关键代码：

```
const canvas = await html2canvas(document.querySelector("video")); // 将视频当前帧
// 作为参数，创建 cavans 对象
const base64 = canvas.toDataURL(); // 得到图片的 base64 数据
let ino = await FileOpt.uploadImage(base64, this.ino); // 上传图片并获得文件的 ino 值
this.content += '\n'; // 将图片
```

链接添加到笔记内容的后面

另一种是在播放器的右侧按钮，点击按钮，系统获取视频当前帧画面，展示在按钮下方，如图 24 所示。点击截图可以对截图进行处理，如截取、绘画、添加文字等，如图 25 所示。将图片处理好后，点击保存按钮，可选择将图片保存到本地还是上传到笔记。值得注意的是，将图片保存到本地时，需要将 base64 数据转换成 Blob 类型。



图 24 截屏按钮位置



图 25 截屏处理界面图

关键代码：

```
var arr = code.split(','), mime = arr[0].match(/:(.*?);/)[1],
bstr = atob(arr[1]), n = bstr.length, u8arr = new Uint8Array(n); // 将 base64 解码
while (n--) { u8arr[n] = bstr.charCodeAt(n); } // 将 base64 转化为 ASCLL 码
let blob = new Blob([u8arr], { type: mime }); // 生成 Blob 对象
```

（2）相关技术

Canvas: Canvas API 提供了一个通过 JavaScript 和 HTML 的<canvas>元素来绘制图形的方式，可以用于动画、游戏画面、数据可视化、图片编辑以及实时视频处理等方面^[25]。与其他图像格式相比，Canvas 是基于矢量绘制的，图像的大小较小；可以用于创建复杂的动画和交互式应用程序；可以通过 JavaScript 动态生成；可以在不同的设备和浏览器上运行，因为他是基于 Web 标准的。

4.3.5 视频标记点

本系统在 markdown 语法的基础上进行了扩展，新增视频标记点语法，可以对视频的某个时间点记录到笔记当中，在预览笔记时，点击标记点，视频会跳转到指定位置，如图所示。在笔记中添加视频标记点有以下好处：提高学习效率，视频标记点可以帮助用户快速定位到视频中的关键内容；便于知识点整理，通过视频标记点，可以将视频中的知识点分门别类地整理，形成完整的学习笔记，方便学习者日后查阅；方便分享与交流，通过视频标记点，可以方便地将自己的学习笔记分享给他人，促进知识交流和共享；提高学习兴趣，通过视频标记点，学习者可以更有针对性地选择自己感兴趣的内容进行学习，提高学习兴趣，激发学习动力。

关键代码：

```
let { currentTime } = document.querySelector("video"); // 获取视频当前播放时间
currentTime = this.formatTime(currentTime); // 格式化时间
this.content += `n- [Description]<time>(${currentTime})\n`; // 添加到笔记中
```

4.3.6 笔记导出

为满足用户的多种需求、增加软件的可用性，本系统支持对笔记进行导出。目前只支持导出为 md 格式和 pdf 格式。其中将笔记导出为 pdf 格式时，需要借助第三方插件 md-to-pdf。首先需要对导出的 pdf 格式进行设置，比如 pdf 样式、页面大小、页边距大小等。得到 pdf 数据流后将其压入 HTTP 流中，客户端再进行下载。

关键代码：

```
let bufferStream = new stream.PassThrough(); // 定义一个双向流  
bufferStream.end(pdf.content); // 将数据流写入  
bufferStream.pipe(res); // 利用管道流压入 HTTP 相应流中
```

4.4 网盘模块设计

网盘模块是本系统的核心模块之一，用户可以通过网盘模块来实现对知识的数字化管理，构建并完善自己的个人知识体系。而数字化知识的便携性、保存完整、检索方便等特点有利于提高用户工作学习效率。本系统的网盘模块主要包含以下几个功能：文件系统的展示、文件/文件夹的更改、文件的上传、最近查看文件的展示等。下面以文件“导数的定义.mp4”为示例，从上传文件到删除文件的整个步骤对网盘模块的使用进行演示，演示步骤如图 26 所示。

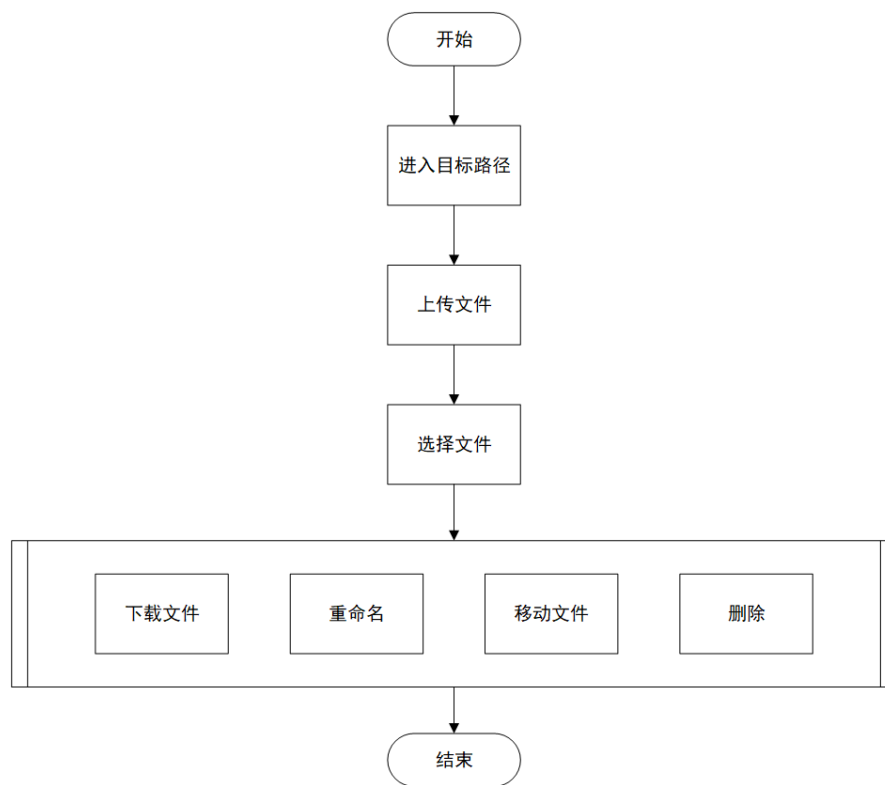


图 26 网盘模块演示步骤

(1) 进入目标路径，用户进入网盘页面，便可以浏览文件系统，如图 27 所示。所有用户的网盘根目录都是“我的网盘”。网盘上方有前进、后退、刷新按钮，并显示当前所在路径。进入“我的网盘>高等数学>第 2 章 导数”目录下，如图 28 所示。



图 27 我的网盘页面

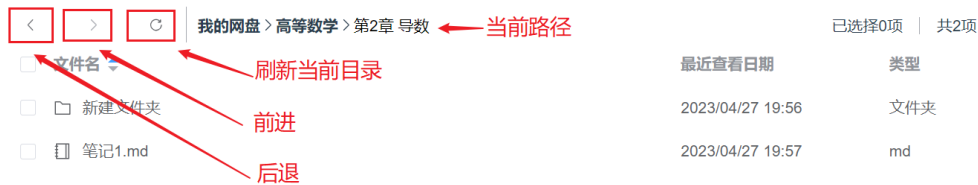


图 28 文件系统操作说明

(2) 上传文件，用户进入到目标文件夹后，点击上传文件按钮，选择本地文件“导数的定义.mp4”进行上传，如图 29 所示。上传后的文件将被展示在当前目录下，如图 30 所示。

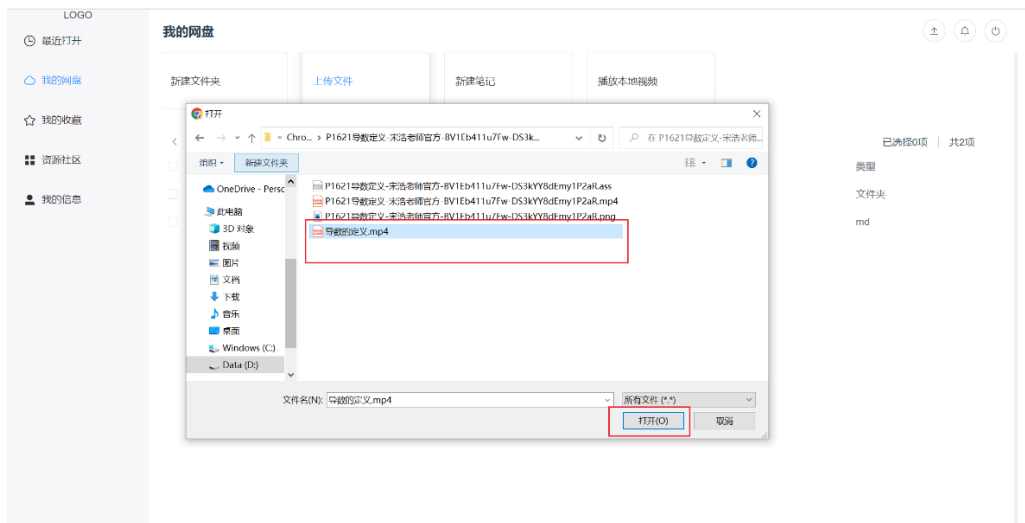


图 29 选择上传文件界面图

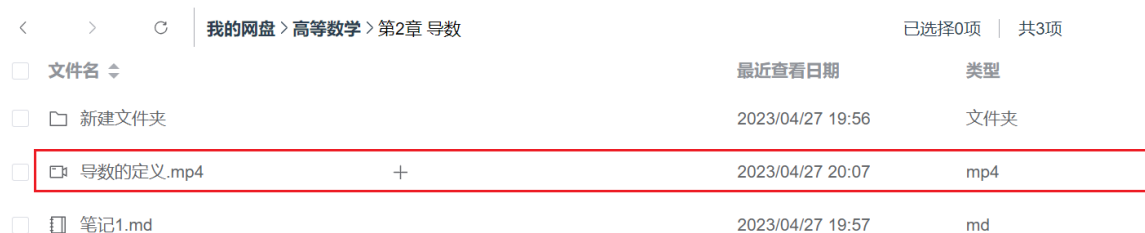


图 30 上传 video.mp4 成功界面图

(3) 选择文件，选择“导数的定义.mp4”文件，文件系统组件上方会出现一排操作按钮，如图 31 所示。

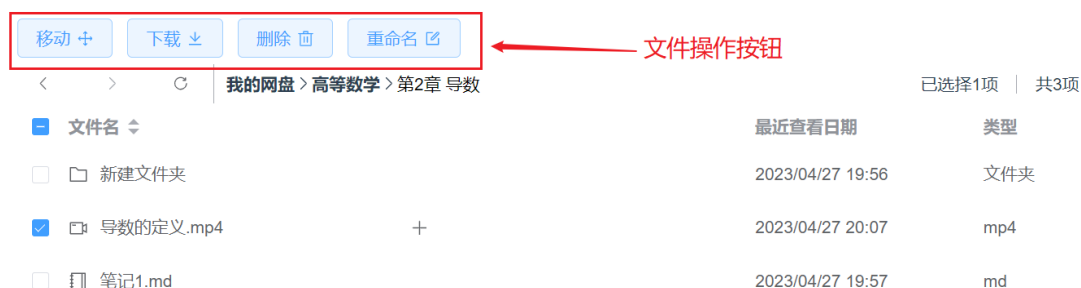


图 31 文件操作按钮界面

(4) 下载文件，选择“导数的定义.mp4”文件，点击下载按钮，浏览器会自动下载文件，如图 32 所示。



图 32 浏览器自动下载文件

(5) 重命名文件，选择“导数的定义.mp4”文件，点击重命名按钮，会出现一个输入框，如图 33 所示。输入文件的新名称“2.1 导数的定义-宋浩”，点击回车，修改名称成功，如图 34 所示。

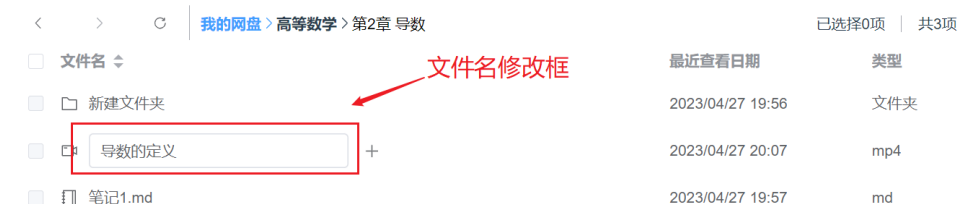


图 33 重命名文件

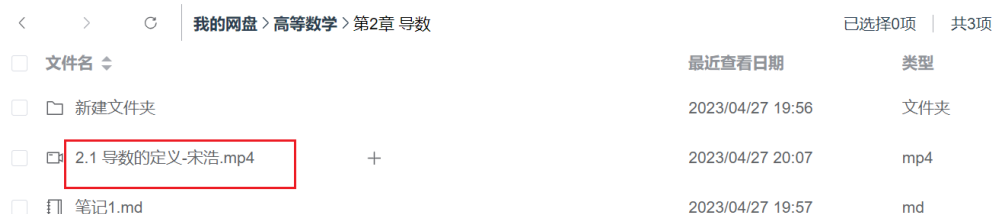


图 34 重命名文件成功

(6) 移动文件，选择“2.1 导数的定义-宋浩.mp4”文件，点击移动按钮，会弹出路径选择器，如图 35 所示。选择目标路径“我的网盘>高等数学>新建文件夹”，点击确定。移动成功后路径“我的网盘>高等数学>新建文件夹”下的目录如图 36 所示。



图 35 路径选择器界面图

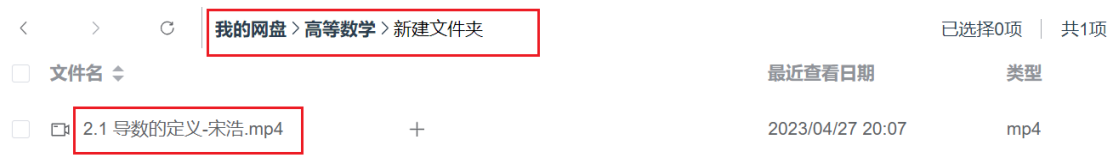


图 36 移动文件成功

(7) 删除文件，选择“2.1 导数的定义-宋浩.mp4”文件，点击删除按钮，会将文件从网盘中彻底删除。

4.4.1 文件系统的展示

(1) 设计思路

和一般的操作系统的资源管理器一样，网盘的文件系统也需要向用户展示文件/文件夹的基本信息，包括文件/文件夹名、最近查看时间和文件类型，如图 28 所示。除此之外，该文件系统对于文件类型为可播放视频 video 的时候，会在文件名后面展示与其相关的笔记，点击笔记或视频会跳转至详情页面；当点击图片类型的文件时，可以对该目录下的图片进行预览。

该文件系统的路径导航功能借鉴了 Windows10 的资源管理器，使用了栈这种数据结构来保存历史路径。当进入某个路径下时，系统会将该路径入栈。点击前进或后退按钮时，会相应地进行入栈和出栈操作。

前进操作核心代码：

```
this.curIndex++; // 修改栈中指针位置
```

```
this.changePaths(this.pathsStack[this.curIndex]); // 根据指针位置，修改当前路径
```

后退操作核心代码：

```
this.curIndex--; // 修改栈中指针位置
```

```
this.changePaths(this.pathsStack[this.curIndex]); // 根据指针位置，修改当前路径
```

文件系统最重要的问题就是路径匹配的问题，作为一款 B/S 架构的应用，解决客户端和服务器的文件系统的路径匹配是十分重要的。文件路径有两种表示方法，相对路径和绝对路径。为了存储系统的可移植性，本系统采用相对路径来匹配文件在服务器上的真实路径。在用户注册时，系统会自动在服务器项目根目录的上一层目录下的 FileCache 下创建一个以用户 uid 命名的文件夹，用户在网盘的所有文件都将存储在该文件夹中。该文件夹即用户在界面上看到的网盘根目录。用户每次进行与路径有关操作时，请求中都会携带相对路径，后端获得相对路径和用户 uid 后通过 node.js 的内置模块 path 便可

以得到文件/文件夹在服务器上的绝对路径。

核心代码：

```
let UPLOAD_DIR = path.resolve(__dirname, homePath, req.auth.uid); // 获取用户网盘根目录的绝对路径
```

```
let dirPath = req.query.path; // 获取客户端发送的相对路径
```

```
dirPath = path.resolve(UPLOAD_DIR, dirPath); // 拼接路径，得到文件的绝对路径
```

（2）相关技术

栈：栈是数据结构中重要的线性结构，是一种特殊的线性表，只允许在表的一端进行插入或删除操作的线性表^[26]。本系统利用栈先进后出的特点，每次进入一个新目录下，都会将路径入栈，路径回退就对应着出栈操作。

path 模块：path 模块是 Node.js 官方提供的、用来处理路径的模块。它提供了一系列的方法和属性，用来满足用户对路径的处理需求。在本系统中 path 模块是必不可少的，通过 path 模块可以很轻松地获得当前代码文件的绝对路径，也可以使用 __dirname 属性将路径字符串进行拼接。

4.4.2 新建/删除文件、文件夹

（1）新建文件、文件夹

由于文件夹的信息不需要写入数据库，所以新建文件夹的实现非常简单，只需要客户端将新建文件夹的相对路径发送给服务器，服务器再根据相对路径得到绝对路径，并在该路径下新建一个文件夹即可；新建文件相对复杂一些，除了需要将相对路径发送到后端，在新建文件成功后，还需要将文件信息保存到数据库当中。由于系统的特殊性，在本系统中，只允许用户新建文件扩展名为 md 的文本文件。新建流程图如图 37 所示。

核心代码：

```
while (fse.existsSync(v_dirPath)) { v_dirPath = dirPath + "(" + ++i + ")"; } // 查询该路径是否已经存在，若存在则在后面加（1）...（n）编号
```

```
fse.mkdirSync(v_dirPath) // 创建文件夹
```

```
fse.writeFileSync(filePath, ""); // 创建文件
```

```
let result = await fileHandler.saveToDB(filePath, req.auth.uid); // 将文件信息写入数据库
```

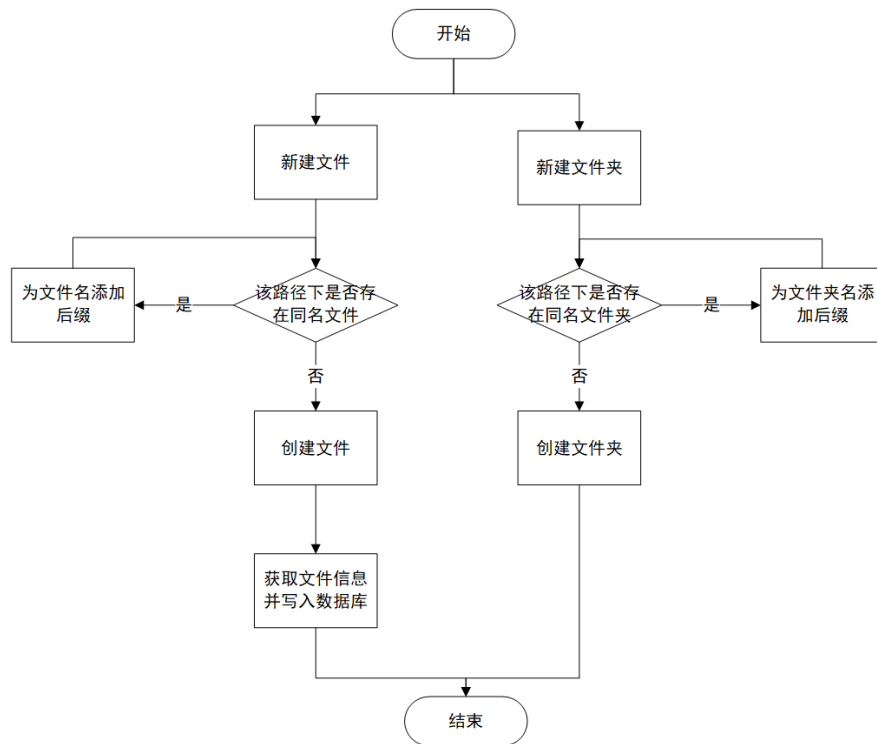


图 37 新建文件、文件夹流程图

(2) 删除文件、文件夹

同新建文件夹，删除空文件夹的操作也非常简单。下面主要讨论删除非空文件夹的情况。可以把文件系统的结构看成树这种数据结构，当需要对树的某个节点进行删除时，往往需要使用递归的方法。同理，本系统在删除非空文件时也使用了递归操作。①首先获取当前路径 `curPath` 下的目录信息，当该文件夹为空时，直接删除该文件夹；②否则遍历目录，根据文件名获得新路径 `newPath`，判断该路径是文件还是文件夹，当该路径为文件时，获取文件的 `ino` 值，根据 `ino` 值删除该文件在数据库中的信息，再将该文件从服务器删除。当该路径为文件夹时，则以此路径再次进行①操作，直至删除所有文件。③删除当前路径 `curPath` 指向的文件夹。删除流程图如图 38 所示。

核心代码：

```

if (file.isDirectory()) { // 当当前路径为文件夹时
    let files = fse.readdirSync(filePath, { withFileTypes: true }); // 获取目录信息
    files.forEach(item => { // 遍历目录
        deleteFile(path.resolve(filePath, item.name)); // 递归调用删除操作
    })
}

```

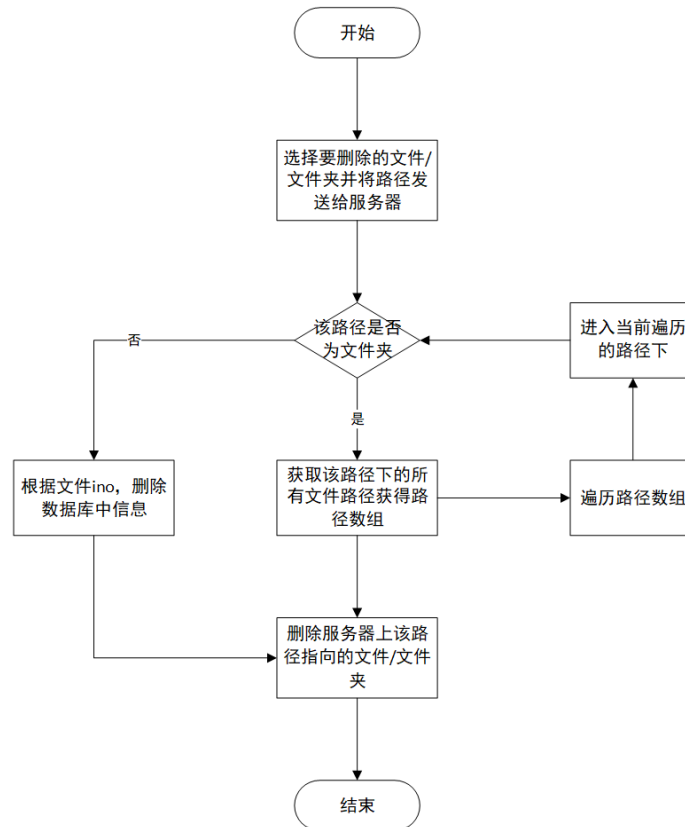


图 38 删除文件、文件夹流程图

4.4.3 重命名/移动文件、文件夹

(1) 重命名文件、文件夹

在文件系统中选中某一个文件或文件夹，点击重命名按钮，即可为文件、文件夹进行重命名，要求输入的新名称不能为空，不能和其他文件名一致，如图 34 所示。当重命名文件时，服务器需要将实际文件的文件名进行修改，同时将数据库中文件信息的绝对路径进行更新。当重命名文件夹时，除了将服务器中对于文件夹的名称修改，还要进行递归操作将该文件夹及其子文件夹下文件在数据库中的文件信息进行更新。

核心代码：

```
fse.renameSync(oldPath, newPath); // 重命名文件/文件夹
```

```
if (file.isDirectory()) { updatePathToDB(newPath); } // 如果该路径为文件夹，需要对目录及其子目录下文件的数据库信息进行更新
```

```
else { fileOpt.rename(file.ino, newName, newPath); } // 如果该路径为文件，则直接对数据库信息进行更新
```

(2) 移动文件、文件夹

在文件系统中选中一个或多个文件/文件夹，点击移动按钮时，会弹出路径选择器，要求选择要移动到的新路径，如图 35 所示。服务器对路径下文件进行过滤，只返回文件夹信息。用户每展开一个文件夹都会向服务器发送一次数据，获取该路径下文件夹信息。和重命名操作一样，移动操作也需要将受本次操作影响的所有文件在数据库当中的信息进行更新。

核心代码：

```
fse.moveSync( oldDirPath, newDirPath); // 移动文件/文件夹
```

```
updatePathToDB(newDirPath); // 对该目录及其子目录下文件的数据库信息进行更新
```

4.4.4 上传文件

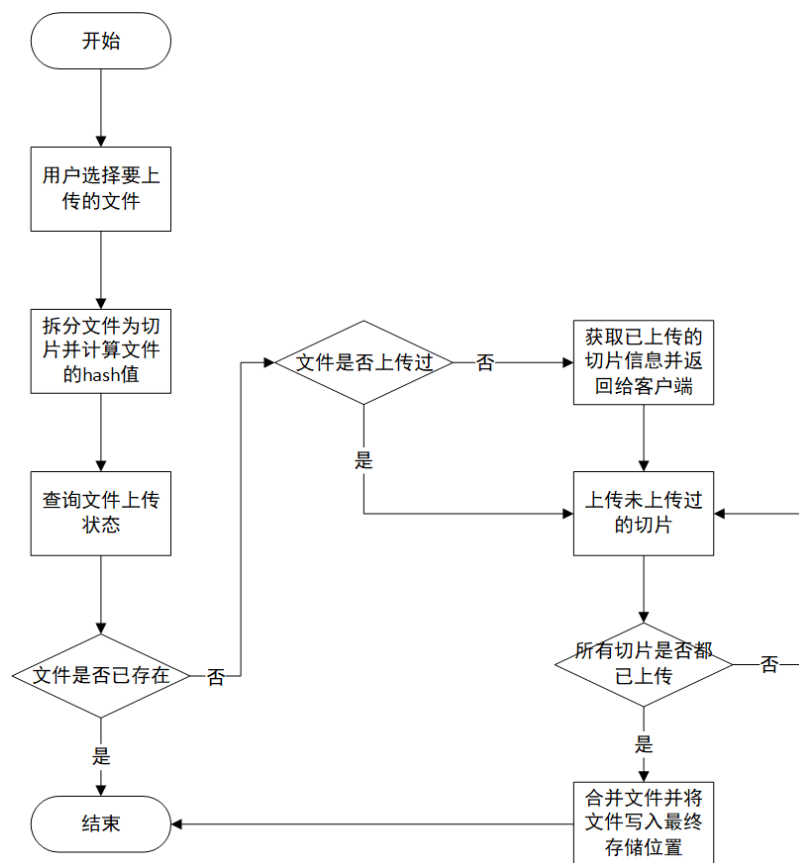


图 39 上传文件流程图

(1) 实现思路

客户端利用 HTML 中的 form 元素和 input 元素，再结合 AJAX 技术可以很容易将

文件传送到服务器。对于普通的图片和几 MB 的文件上传使用上述方法即可完成流畅的上传功能，但 HTTP 协议并不适合处理超大的请求体，文件上传的稳定性存在着很大的问题。一旦大文件的在上传过程出现任何异常，都将导致上传失败，需要重新上传，这非常影响用户的体验感。为了解决这个问题，本系统采用分片上传和断点续传来实现大文件的上传^[27]，步骤主要包括文件切片、计算文件 hash 值、查询文件是否已经上传、上传切片、合并文件，文件上传的流程图如图 39 所示。

①文件切片，在用户选择要上传的文件后，客户端获取文件的二进制内容，然后对其内容进行拆分成一个个 chunk，本系统设定的 chunk 最大大小为 200KB。

核心代码：

```
const maxLen = Math.ceil(this.file.size / size); // 获取切片数
```

```
chunks.push({ index: cur, file: this.file.slice(start, end) }); // 拆分文件压入 chunks 数组
```

②计算 hash 值，使用 MD5 算法根据文件的二进制内容生成一个 128 字节的哈希值。当文件过大时，需要花费很长的时间计算 hash 值，这时候页面不能进行其他操作，所以本系统使用 Web Worker 来计算 hash 值，Worker 开辟了一个新的线程，这使得系统在计算 hash 值的同时，用户可以进行其他操作。

核心代码：

```
this.worker = new Worker(); // 创建 Worker 线程
```

```
this.worker.postMessage({ chunks }); // 传入二进制内容
```

```
this.worker.onmessage = function(e) => {...}; // 计算 hash 值结束调用 onmessage 函数
```

③查询文件是否已经上传，得到 hash 值后，客户端将 hash 值发送给服务器，服务器判断缓存路径下是否存在以 hash 值为名称的文件夹，如果不存在则创建该文件夹。获取文件夹中的已经上传的切片信息并合并成数组，返回给客户端。

关键代码：

```
const dirPath = path.resolve(UPLOAD_DIR, `${hash}`); // 得到缓存路径
```

```
uploadedList = fse.existsSync(dirPath) ? (await fse.readdir(dirPath)).filter(name => name[0] !== '.'): []; // 获取文件夹中所有已经上传的切片
```

④上传切片，客户端根据已经上传的切片信息数组，只将 chunks 数组中未上传的切片进行上传。

⑤合并文件，当所有切片都已经上传后，客户端发起合并请求。服务器将所有切片进行合并，并写入到文件的最终路径，同时在数据库中新增该文件的信息。

核心代码:

```
let chunks = await fse.readdir(chunkDir); // 读取切片
```

```
chunks = chunks.sort((a, b) => a.split('-')[1] - b.split('-')[1]); //切片要按顺序合并, 因此需要排序
```

(2) 相关技术

Web Worker: 众所周知 JavaScript 是一门单线程的语言, 当遇到需要进行大量计算的场景时, JS 线程往往会被长时间阻塞, 甚至造成页面卡顿, 影响用户体验。而 HTML5 标准中的 Web Worker 的出现使得 JS 实现多线程成为了可能。Web Worker 定义了一套 API, 允许在 JS 线程之外开辟新的 Worker 线程, 并将一段 js 脚本运行其中^[28]。因为是独立的线程, Worker 线程和 JS 线程可以同时运行, 互不阻塞。所以在本系统中将 hash 值的计算任务交给 Worker 线程处理, 当 Worker 线程计算完成, 再将结果返回给 JS 线程。

4.4.5 下载文件

(1) 实现思路

下载文件相较上传文件要简单得多, 由于网盘中的每个文件信息都会被存储在数据库中, 在下载文件时, 客户端只需要向服务器提供文件的 ino 值即可。服务器根据 ino 值在数据库中获取文件的绝对路径, 再使用 fs-extra 模块读取文件并通过管道流压入 HTTP 通道。客户端接收响应的 Blob 数据, 生成虚拟地址, 再交由浏览器进行下载。值得注意的是, 客户端在响应请求时, 要将响应头的 Content-Type 字段的值设为“application/octet-stream;charset=utf-8”, 以提醒客户端本次请求的响应数据将是一个文件流。

关键代码:

```
const blob = new Blob([res]); // 将响应当数据处理为 Blob 类型
```

```
var url = window.URL.createObjectURL(blob); // 创建一个 URL 对象指向 blob 流
```

```
var a = document.createElement("a"); // 创建 a 标签进行下载
```

```
a. href = url;
```

(2) 相关技术

window.URL: 该接口用于解析、构造、规范化和编码 URL。通常, 通过在调用 URL 的构造函数时将 URL 指定为字符串或提供相对 URL 和基本 URL 来创建行的 URL 对象。URL.createObjectURL() 静态方法会创建一个 DOMString, 其中包含一个表示参数中给出的对象的 URL。这个 URL 的生命周期和创建它的窗口中的 document 绑定, 这个新

的 URL 对象表示指定的 File 对象或者 Blob 对象。

4.4.6 最近查看文件的展示

在一段时间内，用户可能只对某一个或几个文件有频繁查看的需求。为了提高用户体验，减少用户的检索时间，本系统增加了最近查看文件的展示功能，如图 40 所示。这还有利于用户对知识的管理，提高用户效率。在实现上，本系统设置了一个最近查看文件表，专门用来记录用户最近查看的文件。该表主要由用户 uid 字段和文件数组 files 字段构成。files 中记录着最近查看文件的 ino 值，按最近查看时间降序排序。

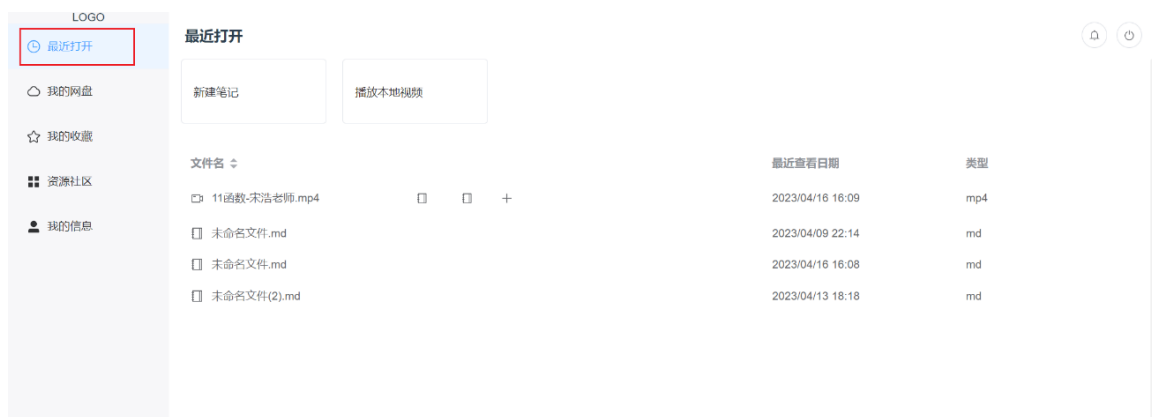


图 40 最近查看文件界面

关键代码：

```
let index = files.findIndex(item => item === ino); // 寻找要新增 ino 在 files 的索引
if (index !== -1) { files.splice(index, 1); } // 如果 files 中存在 ino，需要先将其删除
files.unshift(ino); // 数组开头插入 ino
```

4.5 资源社区模块设计

资源社区模块和我的收藏模块都是本系统的扩展模块，从产品的角度来看，有以下作用和好处：促进用户互动和交流，用户可以在这里分享自己的体验、技巧和心得，与其他用户交流互动，提高用户体验感；提供更多资源和内容，丰富软件的功能和内容，提高软件的使用价值和用户满意度；提高用户粘性和忠诚度，资源社区模块可以为用户提供更多的价值和服务，从而降低用户流失率^[29]。

4.5.1 社区帖子的展示与搜索

资源社区的内容都展示在资源社区页面，每个帖子会展示关键信息，标题、内容、

标签、点赞数、收藏数等，如图 41 所示。其中由于帖子内容是以 md 格式存储的文本文件，在资源社区页面获取每一个帖子的内容的话，会造成资源的大量浪费，因为只需要向用户展示帖子内容的前几行。为了解决这个问题，在服务器中只读取帖子内容的前 1KB 内容。点击帖子会跳转到帖子详情页面，可以查看帖子关联的视频和笔记。在帖子详情页面用户可以对帖子进行点赞和收藏。



图 41 资源社区界面

对于资源社区而言，如何为用户精准推送用户想要的资源与内容是十分重要的。一般有两种方式，一是进行大数据推送，收集用户行为数据，将分析出的有价值的数据和信息推送给用户；二是提高搜索命中率，根据用户在搜索时提供的关键字更加准确地返回搜索结果。在本系统中为每个帖子都增加了 tag 标签，用户可以选择自己所需要的标签对资源社区的内容进行筛选展示，如图 41 左侧标签栏所示。

读取文件前 1KB 数据关键代码：

```
fse.open(filePath, 'r+', (err, fd) => {  
    const content = Buffer.alloc(size); //创建一个 buffer 类型的变量 content 来存储读取到的数据  
    fse.read(fd, content, 0, 1024, null, (err, bytesRead, buffer) => {  
        resolve(content.toString('utf-8'));  
    });  
});
```

```
})// 获取文件前 0-1023 个字节
```

```
})
```

4.5.2 发布帖子

每个合法用户都可以在资源社区中发布内容。点击发布内容按钮，客户端会向服务器请求一个_id，该_id即为当前编辑的帖子的_id。接着初始化页面数据后，用户便可以在该页面填写帖子内容，页面 UI 如图 42 所示，发布流程如图 43 所示。

帖子表单包括标题、标签、封面、视频、内容这几个表单项。其中用户至少为帖子添加一个标签，用户在添加标签时需要先在数据库中搜索，若没有该标签项，点击新增按钮，可以将新增的标签写入数据库中；封面的上传和 4.2.1 基本设置中头像的上传原理一致，在这里就不多做阐述；添加视频时可以选择本地视频也可以选择已经上传到网盘的视频，选择本地视频时需要先等待本地视频被上传到网盘。选择网盘视频时，与视频相关联的笔记也会被展示在帖子详情当中。帖子内容的编辑复用了 4.4.3 笔记编辑与保存的功能。

图 42 发布帖子界面图

当所有表单项被用户正确的填写完整，点击立即发布按钮，帖子状态被置为 1，可在我的发布和资源社区中搜索到；当用户想暂存帖子内容，点击暂时保存，当前帖子将被保存为草稿。用户下次进入发布内容页面会直接显示未完成的草稿内容；当用户想要

丢弃当前编辑的帖子，点击取消发布，帖子的所有信息会从数据库中删除。

关键代码：

```
await this.$axios.post("/invitation/publish", {_id, isNew, opt: { title, tags }}); // 发布帖子
```

```
await this.$axios.post("/invitation/save", {_id, opt: { title, tags }}); // 暂存草稿
```

```
await this.$axios.delete('/invitation/deleteInvitation', { data: {_id, uid }}); // 删除草稿
```

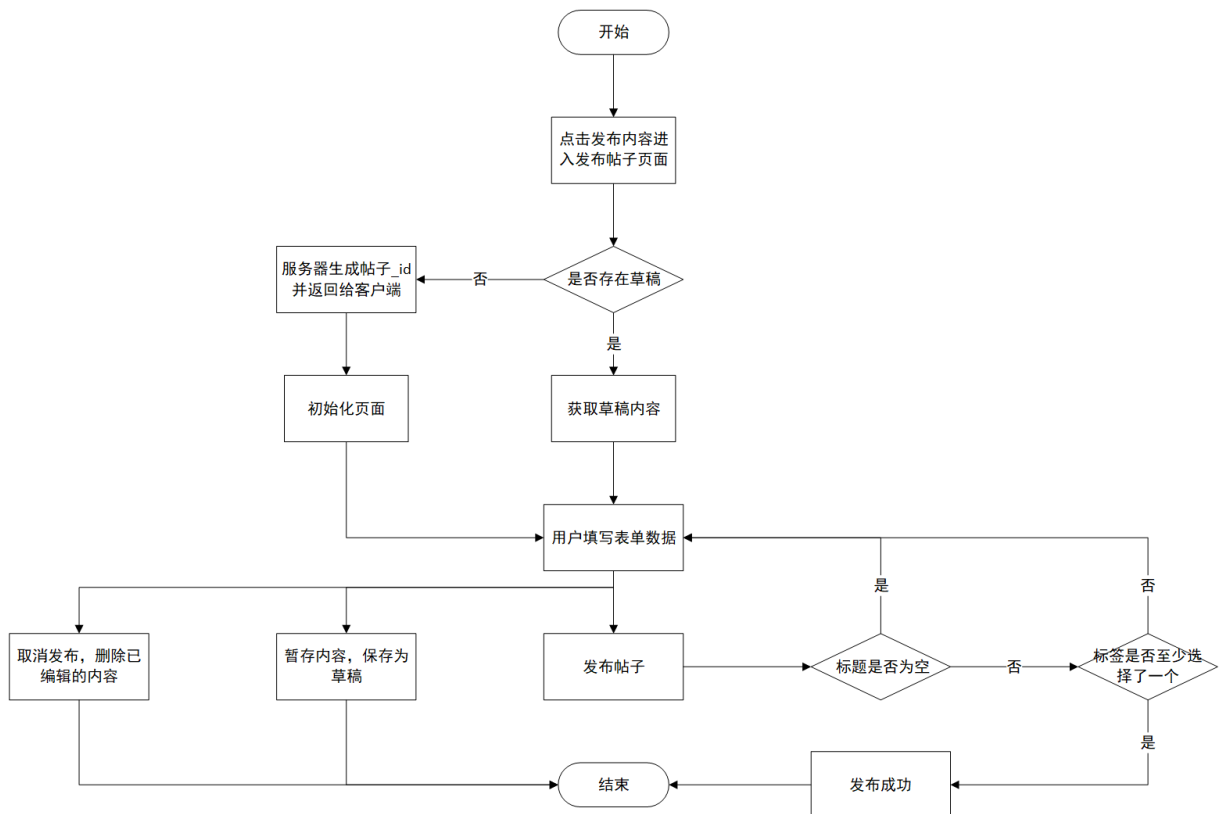


图 43 发布帖子流程图

4.5.3 管理帖子

用户可以在我的发布页面查看管理所有曾经发布过的帖子内容，支持用户按标签进行搜索，也支持用户按发布时间和点赞数进行排序。点击某个帖子，会跳转到帖子详情页面，在该页面的头部会出现编辑和删除的按钮，如图 44 所示。点击编辑按钮，会跳转到发布内容页面要求用户对帖子进行编辑，此时不支持暂存为草稿；点击删除按钮会将帖子的所有信息删除且不支持恢复操作。



图 44 帖子详情页面

4.6 我的收藏模块设计

收藏模块是资源社区必不可少的辅助模块，它可以方便用户整理资源，方便以后查看和使用；增加社区内容的曝光度，用户收藏的资源会在收藏夹中展示，同时也会在社区中被推荐给其他用户；促进用户互动，增加社区活跃度；为产品运营提供数据支持，通过收藏功能，可以收集用户的偏好和需求，优化产品功能和内容。

4.6.1 收藏帖子

在帖子详情页面，用户点击收藏按钮时，会弹出收藏夹选择器，如图 45 所示。收藏夹选择器展示用户所有的收藏夹。用户还可以根据需要新建收藏夹。

收藏夹选择器除了在此处引用，还会在管理收藏夹中被应用。为了增强代码的可复用性和易修改性，系统将收藏夹选择器单独抽离为一个组件，并命名为 `CollectionList.vue`。`Collection` 作为一个帖子详情页面的一个子组件，当用户点击收藏按钮时，会显示该组件。在前端模块化中，最重要的就是模块间父子组件的通信，`Vue` 中对于父子组件通常使用 `props` 和 `$emit/$on` 进行通信。在这里我们使用 `props` 传递参数控制 `CollectionList` 组件的展示与隐藏，当用户选择好收藏夹后，`CollectionList` 需要将被选择收藏夹的 `_id` 传递给父组件，以便进行后续操作。在这里 `CollectionList` 使用 `$emit` 通知父组件，父组件则使用 `$on` 等待通知。

关键代码：

```
this.$emit("selectCollection", this.currentCol); // 将选好的收藏夹信息传递给父组件  
this.shadowDialog(); // 隐藏收藏夹选择器窗口
```



图 45 收藏夹选择器

4.6.2 管理收藏夹、收藏内容

用户的收藏内容都会在我的收藏页面展示，左侧展示收藏夹信息，右侧展示收藏夹的详细内容，如图所示。

点击左侧下方管理收藏夹按钮，将弹出收藏夹选择题，可对收藏夹进行管理。包括新建收藏夹和删除收藏夹。选择想要删除的收藏夹，点击删除按钮可删除收藏夹中所有内容。点击新建收藏夹按钮，弹出表单窗口，要求用户对新建收藏夹进行设置，包括收藏夹名称、收藏夹描述、收藏夹图标、收藏夹颜色。

用户可以对收藏的帖子进行删除和移动，其中移动操作也需要借助收藏夹选择器。当用户收藏的某个帖子被作者删除后，页面会提示用户是否需要删除无效收藏。



图 46 我的收藏页面

4.7 本章小结

本章为系统的详细设计，主要介绍了关键模块的关键功能实现和使用到的相关技术，

有登录注册模块、个人信息模块、视频笔记模块、网盘模块、资源社区模块、我的收藏模块。其中登录模块中的登录状态验证功能使用了 `token` 进行状态验证；个人信息模块中的修改头像功能需要上传图片文件；视频笔记模块中笔记编辑与保存、视频画面截取等多个功能都是本系统的核心功能之一；网盘模块主要使用了第三方依赖包 `fs-extra` 来实现；资源社区模块和我的收藏模块更侧重于介绍其中的业务逻辑；

5 系统测试与分析

5.1 测试概述

软件测试是在软件开发过程中，通过对软件系统进行各种测试，发现软件系统的缺陷和问题，以保证软件系统的质量和稳定性的过程。它的目的是为了发现软件系统中的错误、缺陷和问题，提高软件系统的质量和可靠性，确保软件系统能够按照预期的功能和性能要求正常工作^[30]。在软件测试中，我们会根据软件系统的需求规格说明和功能规格说明书进行测试，以确保软件系统的功能和用户需求是否满足，同时也会对软件系统的内部结构和源代码进行测试，以确保软件系统的可靠性和安全性。软件测试可以应用于软件开发的各个阶段，包括需求分析、设计、编码、测试和维护等^[31]。

5.2 功能测试

5.2.1 登录注册功能测试

（1）用户登录

用户登录是用户使用系统的前置条件，需要用户输入账号密码进行验证，用户登录功能测试用例表如表 8 所示。

表 8 用户登录测试用例表

项目名称	项目内容
测试用例描述	用户登录
用例前置条件	无
输入流	用户账号、用户密码
预期结果	表单提交成功，账号密码验证成功，返回客户端用户信息及 token，进入系统主页面
用户操作步骤	1.进入登录页面；2.填写完整表单；3.点击登录按钮
测试结果	符合预期结果，用户登录成功

（2）用户注册

用户在进行注册时，需要设置账号和密码，且账号不能已被注册。用户注册功能测试用例表如表 9 所示。

表 9 用户注册测试用例表

项目名称	项目内容
测试用例描述	用户注册
用例前置条件	无
输入流	用户账号、用户密码、确认密码
预期结果	用户注册成功，关闭注册窗口
用户操作步骤	1.进入登录页面；2.点击用户注册按钮；3. 用户填写完整表单；4.点击注册按钮
测试结果	符合预期结果，用户注册成功

测试总结：经过对登录注册功能测试发现，用户登录注册功能正常。

已达到的功能：①用户在登录注册表单进行输入时，输入非法时界面会正确提示错误信息；②用户注册成功后，可以立即登录。

暂未实现的功能：①用户在多次输入密码失败后，没有登录冷却时间。

5.2.2 视频笔记功能测试

（1）视频跳转播放

对于视频播放，用户可以通过点击或滑动进度条，或者点击笔记中标记点来实现视频的跳转播放。视频跳转播放用例表如表 10 所示。

表 10 视频跳转播放测试用例表

项目名称	项目内容
测试用例描述	视频跳转播放
用例前置条件	用户已登录，已打开视频播放界面
输入流	视频的时间点
预期结果	视频进度跳转成功，并且不改变视频的播放暂停状态
用户操作步骤	1.点击进度条； 1.滑动进度条； 1.点击笔记文件中视频标记点
测试结果	符合预期结果，视频跳转播放成功

（2）笔记自动保存

在用户编辑笔记时，系统会每间隔 10s 对笔记进行一次自动保存。笔记自动保存的用例表如表 11 所示。

表 11 笔记自动保存测试用例表

项目名称	项目内容
测试用例描述	笔记自动保存
用例前置条件	用户已登录，已打开笔记编辑界面
输入流	笔记文字内容
预期结果	10s 后笔记保存成功
用户操作步骤	1.编辑笔记导致笔记发生更改
测试结果	符合预期结果，笔记保存成功

（3）视频截图保存至本地

用户可以通过点击视频播放器右侧的截屏按钮，对视频画面进行截图并保存至本地。视频截图保存至本地功能测试用例表如表 12 所示。

表 12 视频截图保存至本地测试用例表

项目名称	项目内容
测试用例描述	视频截图保存至本地
用例前置条件	已打开视频播放界面
输入流	图片名称、图片类型、图片大小
预期结果	展示视频截图，浏览器下载图片到本地
用户操作步骤	1.点击播放器右侧截屏按钮；2.点击截图；3.编辑处理截图；4.点击保存按钮；5.设置图片信息；6.点击按钮保存至本地
测试结果	符合预期结果，截图保存成功

（4）视频截图上传并写入笔记

支持用户点击按钮快速将视频画面保存为截图并将链接写入笔记中，测试用例表如

表 13 所示。

表 13 视频截图上传并写入笔记测试用例表

项目名称	项目内容
测试用例描述	视频截图上传并写入笔记
用例前置条件	已打开笔记编辑界面，已打开视频播放页面
输入流	无
预期结果	截图上传成功，笔记中插入图片链接且笔记预览中可以正常显示图片
用户操作步骤	1.点击笔记编辑器上方的截图按钮
测试结果	符合预期结果，截图上传成功，写入笔记成功

（5）笔记导出

用户可将笔记导出为 md 文件和 pdf 文件，笔记导出功能测试用例表如表 14 所示。

表 14 笔记导出测试用例表

项目名称	项目内容
测试用例描述	笔记导出
用例前置条件	用户已登录，已打开笔记编辑界面
输入流	无
预期结果	浏览器下载笔记对应的 md 文件、pdf 文件到本地成功
用户操作步骤	1.点击笔记编辑器上方的导出按钮；2.选择导出为 md 文件/pdf 文件
测试结果	符合预期结果，文件下载成功

测试总结：经过测试发现视频笔记功能可以正常使用。

已达到的功能：①视频可以正常跳转播放，点击视频标记点和视频进度条都可以改变视频时间点；②笔记自动保存功能正常，修改笔记 10s 后一定会保存一次笔记；③截

图的上传和保存功能正常，截图可以正常上传到网盘并添加到笔记内容中，也可以下载到本地；④笔记导出功能正常，导出为 md 和 pdf 格式都可以正常下载到本地。

5.2.3 网盘功能测试

（1）文件系统的展示

根据文件夹在客户端的相对路径，将其转化服务器的绝对路径，查找该路径下的文件信息从而展示在客户端界面。文件系统的展示用例表如表 15 所示。

表 15 文件系统的展示测试用例表

项目名称	项目内容
测试用例描述	文件系统的展示
用例前置条件	用户已登录
输入流	文件夹的相对路径
预期结果	页面展示该文件夹下所有文件目录
用户操作步骤	1.进入我的网盘页面；2.选择要进入的文件夹（可选）
测试结果	符合预期结果，文件系统展示成功

（2）上传文件

文件的上传在代码上实现比较复杂，但对用户来说只要达到其功能预期即可，表 16 是上传文件的功能测试用例。

表 16 上传文件测试用例表

项目名称	项目内容
测试用例描述	上传文件
用例前置条件	用户已登录，已在我的网盘页面下
输入流	一个或多个本地文件
预期结果	文件上传成功，并且能在文件系统中展示
用户操作步骤	1.点击上传文件菜单；2.选择想要上传的一个或多个文件；3.点击确定
测试结果	符合预期结果，上传文件成功，展示成功

（3）下载文件

文件下载的是用户常常要使用的功能之一，选择网盘中一个或多个文件，点击下载按钮进行测试文件是否下载成功。下载文件功能测试用例表如表 17 所示。

表 17 下载文件测试用例表

项目名称	项目内容
测试用例描述	下载文件
用例前置条件	用户已登录，已在我的网盘页面下
输入流	一个或多个文件的相对路径
预期结果	浏览器自动下载文件并下载成功
用户操作步骤	1.选择要下载的一个或多个文件；2.点击下载按钮
测试结果	符合预期结果，文件下载保存到本地成功

（4）新建文件、文件夹

新建文件、文件夹测试用例表如表 18 所示

表 18 新建文件、文件夹测试用例表

项目名称	项目内容
测试用例描述	新建文件、文件夹
用例前置条件	用户已登录，已在我的网盘页面下
输入流	当前文件夹的相对路径
预期结果	成功创建文件、文件夹，并能成功展示在文件系统中
用户操作步骤	1.点击新建文件夹 1.点击新建文件，选择文件夹路径
测试结果	符合预期结果，文件、文件夹新建成功

（5）删除文件、文件夹

用户删除在界面文件、文件夹后服务器中对应的文件、文件夹也会被删除，其测试

用例表如表 19 所示。

表 19 删除文件、文件夹测试用例表

项目名称	项目内容
测试用例描述	删除文件、文件夹
用例前置条件	用户已登录，已在我的网盘页面下
输入流	一个或多个文件、文件夹的相对路径
预期结果	删除文件、文件夹成功
用户操作步骤	1.选择一个或多个文件、文件夹；2.点击删除按钮
测试结果	符合预期结果，文件、文件夹删除成功

（6）重命名文件、文件夹

重命名文件、新建文件夹测试用例表如表 20 所示。

表 20 重命名文件、文件夹测试用例表

项目名称	项目内容
测试用例描述	重命名文件、文件夹
用例前置条件	用户已登录，已在我的网盘页面下
输入流	文件、文件夹的原相对路径，新相对路径
预期结果	文件、文件夹重命名成功，并能在文件系统中展示新名称
用户操作步骤	1.选择要重命名的文件、文件夹；2.点击重命名按钮；3.输入新名称；4.按下回车或让输入框失去焦点
测试结果	符合预期结果，文件、文件夹重命名成功

（7）移动文件、文件夹

移动文件、新建文件夹测试用例表如表 21 所示

表 21 移动名文件、文件夹测试用例表

项目名称	项目内容
测试用例描述	移动名文件、文件夹
用例前置条件	用户已登录，已在我的网盘页面下
输入流	一个或多个文件、文件夹的相对路径，目标文件夹的相对路径
预期结果	文件、文件夹移动成功，并能在文件系统展示
用户操作步骤	1.选择要移动的一个或多个文件、文件夹；2.点击移动按钮；3.选择要移动到的文件夹；4.点击确定按钮
测试结果	符合预期结果，文件、文件夹移动成功

测试总结：经过对网盘功能的测试发现网盘功能基本能正常使用

已达到的功能：①文件系统展示正常，并且前进、后退、刷新操作能正常使用；②新建文件/文件夹功能正常，新建成功后可以展示在文件系统中；③重命名/移动文件功能正常，操作后可以正常打开、编辑文件；④删除文件功能正常，且删除后不会影响到其他关联文件。

异常功能：①上传文件功能异常，上传体积小的文件时功能正常，但上传大文件时页面可以会发生卡死的情况。

5.2.4 资源社区功能测试

（1）筛选社区帖子

用户通过选择标签来筛选社区帖子，从而获得自己感兴趣的、需要的内容。筛选社区帖子功能测试用例表如表 22 所示。

表 22 筛选社区帖子测试用例表

项目名称	项目内容
测试用例描述	筛选社区帖子
用例前置条件	用户已登录，已打开资源社区页面
输入流	选择的标签

预期结果	页面展示与用户选择的标签相关的帖子
用户操作步骤	1.点击选择页面右侧标签选择栏中的标签项， 可多选；2.点击确认
测试结果	符合预期结果，页面只展示与标签相关的内容

（2）发布帖子

每个合法用户都可以在资源社区发布内容,发布帖子功能测试用例表如表 23 所示。

表 23 发布帖子测试用例表

项目名称	项目内容
测试用例描述	发布帖子
用例前置条件	用户已登录，已打开资源社区页面
输入流	帖子标题、帖子标签、帖子封面、相关视频、 帖子内容
预期结果	帖子发布成功，资源社区中可以搜索到刚发布的帖子
用户操作步骤	1.点击页面上方发布帖子按钮；2.填写发布帖子 表单；3.点击立即发布按钮
测试结果	符合预期结果，帖子发布成功

（3）点赞、收藏帖子

用户可以为喜欢的帖子进行点赞，还可以将其收藏到指定的收藏夹内。点赞、收藏帖子功能测试用例表如表 24 所示。

表 24 点赞、收藏帖子测试用例表

项目名称	项目内容
测试用例描述	点赞、收藏帖子
用例前置条件	用户已登录，已打开帖子详情页面
输入流	无

预期结果	点赞成功；收藏成功，并且可以在我的收藏页面查看
用户操作步骤	1.点击页面上方点赞按钮； 1.点击页面上收藏按钮；2.选择目标收藏夹； 3.点击确认按钮
测试结果	符合预期结果，点赞、收藏帖子成功

（4）新建收藏夹

用户在管理和选择收藏夹时可以新建一个收藏夹，需要用户对收藏夹进行一些设置。新建收藏夹测试用例表如表 25 所示。

表 25 新建收藏夹测试用例表

项目名称	项目内容
测试用例描述	新建收藏夹
用例前置条件	用户已登录
输入流	收藏夹名称、收藏夹描述、收藏夹图标、收藏夹颜色
预期结果	新建收藏夹成功，可以在收藏夹列表中显示
用户操作步骤	1.我的收藏页面的管理收藏夹按钮或者帖子详情页面的收藏按钮；2.在弹出的收藏夹选择器中点击新建收藏夹；3.填写表单；4.点击确认按钮
测试结果	符合预期结果，新建收藏夹成功

（5）编辑、删除帖子

用户可以对自己发布的帖子进行重新编辑和删除操作，编辑、删除帖子功能测试用例表如表 26 所示。

测试总结：经过测试资源社区功能能正常使用。

已达到的功能：①筛选帖子功能正常，用户选择想要的标签后可以获得相关的帖子信息；②发布、编辑、删除帖子功能正常，用户可以正常编辑发布帖子，当用户输入不

合法时，界面会正确显示错误信息。用户删除帖子后其相关信息也会被删除；③点赞、收藏帖子功能正常，收藏帖子的可以在收藏夹中展示；④新建收藏夹功能正常，新建的收藏夹会以正确的样式展示在界面。

表 26 编辑、删除帖子测试用例表

项目名称	项目内容
测试用例描述	编辑、删除帖子
用例前置条件	用户已登录，已进入我的发布页面
输入流	无
预期结果	跳转至编辑页面；帖子删除成功
用户操作步骤	1.点击帖子进入帖子详情页面；2.点击编辑按钮 /删除按钮；
测试结果	符合预期结果，编辑/删除帖子成功

5.3 压力测试

压力测试是系统测试非常重要的一个环节。它可以帮助开发者评估系统在高负载、高并发等极端情况下的性能表现，从而提高系统的可靠性和稳定性。具体来说，压力测试的重要性体现在以下几个方面：①发现性能问题；②确定系统容量；③提高用户体验。本系统使用了 Apifox 自动化测试工具进行压力测试，以获取媒体资源接口 /media/getMedia 为例，分别进行了 10s 内 500 个线程、1000 个线程和 2000 个线程并发的压力测试，测试结果如表 27 所示。

表 27 压力测试结果表

编号	线程数	吞吐量	失败数	异常率
1	500	18.3/sec	4	0.80%
2	1000	54.0/sec	9	0.90%
3	2000	25.2/sec	17	0.85%

(1) 当 500 个线程并发时，总失败数为 4，即系统异常率仅为 0.80%。几乎可以忽略不计，可认定在该环境下系统能够正常工作。

(2) 当 1000 个线程并发时，总失败数为 9，即系统异常率为 0.90%。系统能够在该环境下正常工作。

(3) 当 2000 个线程并发时，总失败数为 17，即系统异常率为 0.85%。系统能够在该环境下正常工作。

经过上述几轮压力测试，可以认定本系统性能较好，能够在高压情况下正常运行，具有较高的可靠性。

5.4 测试总结

本次测试主要使用了黑盒测试的方法，对于系统的功能进行完整的测试，包括用户登录注册、网盘功能、视频笔记功能、资源社区功能。从测试结果来看，本系统的基本功能能够正常运行，但也发现了一些问题。首先，在网盘功能中，当用户上传的文件过大时，会造成页面阻塞，用户无法进行其他操作。经过排查后发现是文件太大时，系统计算文件 hash 值需要很长时间，页面会陷入等待状态。所以我们将 hash 值的计算移入 Worker 进程中，解决了页面阻塞的问题。其次，在视频笔记功能中，系统仅支持 mp4、ts 等常见格式视频的播放。此外，本系统还存在一些小问题，但均不影响正常用户的正常使用。

本次测试也对系统进行了压力测试，测试结果表明系统性能较好，能够在高并发的环境下正常运行，基本满足用户的日常需求。

软件测试对于软件开发具有重要意义，它贯穿了软件的整个生命周期。通过软件测试可以提高软件质量，减少软件系统出现问题的概率；可以及时发现开发的问题和改进点，不断优化软件开发流程，提高软件开发的效率和可控性。

6 总结

作为一名大学四年被疫情填满三年的大学生，笔者经历过一整个学期的网课生活，深知网课学习存在的精神难以集中，学习效率低，课后复盘困难等问题。虽然现在疫情已经退去，也很难长时间都待在家里上网课，但是视频教育仍然是许多人学习知识的重要方式。在学习过程中，记笔记是一项重要的学习策略，能够帮助学习者理解和记忆知识，促进新知识与已有知识的整合，且便于学习者在后期对知识进行回顾和巩固。但是目前市场上并没有专门针对视频方向的笔记工具。基于此背景，本人和导师经过讨论，产生了本课题。

本系统基于 B/S 架构，使用 Vue 框架构建前端内容，使用 Node.js + Express.js 搭建服务器，数据库选用文档型数据库 MongoDB。本系统的主要功能如下：①用户个人账户管理，允许用户注册登录并对个人信息进行修改；②视频播放与笔记编辑预览，支持播放常见视频格式，笔记内容支持 Markdown 语法，笔记编辑支持快速截取视频画面，笔记预览支持视频标记点控制视频播放；③个人知识体系管理，网盘支持上传、下载文件、管理文件等操作；④用户知识分享，用户可以在资源社区发布内容，也可以查看收藏其他用户发布的内容，促进知识传播。

现在市场上流行的笔记类工具，如 OneNote、有道云笔记、腾讯文档、谷歌文档等。它们大都更专注丰富文字笔记内容，而忽略了笔记作用类型的扩展。本系统的笔记编辑工具专注于视频这一细分领域，通过视频标记点将笔记和视频连接起来，用户在复习笔记内容时能够快速定位到视频内容，以加深用户对知识点的印象，提高用户学习效率。

我们应该辩证地看待事物，除了上述实现的功能和本系统的优点外，目前系统还存在着许多可优化的地方，比如目前笔记导出只支持导出为 md 文本格式和 pdf 格式。支持导出格式越多意味着软件的可用性就越高，在以后的优化中，应该支持导出为多种格式；系统支持播放网盘视频和本地视频，但是用户可能有为第三方视频平台做笔记的需求，这就需要支持外部链接的解析，获取并播放外部视频。其他可优化的地方就不多做阐述，一款优秀的软件绝不是一朝一夕就能完成的，需要反复地优化修改。在以后的日子里，随着本人的技术能力的提高，一定不断优化修改本系统，让它成为一款优秀的软件。

参考文献

- [1] 徐雪松,彭雅洁,陈晓红,肖冬,王影捷,颜楚涵.基于粒计算的突发性复合疫情风险评估模型构建[J].系统工程理论与实践,2023,43(02):583-597.
- [2] 木本荣,杨艺,刘文雯,饶安阳,王海.后疫情时代高等教育教学模式思考[J].成都中医药大学学报(教育科学版),2021,23(04):26-29+72.
- [3] 孙士奇,吴善禄.新冠疫情下大学生在线学习效果评价研究[J].网络安全技术与应用,2021(08):101-103.
- [4] 张志杰,段昕彤,王艳霞.疫情防控背景下地方高校学生线上学习满意度调查分析[J].长春师范大学学报,2022,41(09):165-168.
- [5] Tian N , Tsai S B . An Empirical Study on Interactive Flipped Classroom Model Based on Digital Micro-Video Course by Big Data Analysis and Models[J]. Mathematical Problems in Engineering, 2021, 2021:11-22.
- [6] Hurst C . Digital Demise: Does Digital Note-Taking Hinder or Help Learning for the Millennial. 2017,16(5):24-30.
- [7] 苏媛欣.数字化笔记在个人知识管理中的应用[J].知识管理论坛,2015(01):35-41.
- [8] 乔迁,唐绮萱.DIKW 模型在笔记工具类产品设计中的应用与未来方向[J].中国信息化,2019,05:37-39.
- [9] 辛超,乔子健,孙艳春.一种面向 Chrome 浏览器的视频云笔记插件[J].计算机科学,2017,44(04):60-65+89.
- [10] 代明鸿.印象 VS 有道 笔记里的战争[J].商界(评论),2015(11):104-107.
- [11] 刘恩卿.电子笔记本 OneNote 在综合实践课中的应用——以“寻访家乡的传统文化”为例[J].实验教学与仪器,2020,37(Z1):85-86.
- [12] 杨怡晨,亢军贤,白博,赵美映.基于 B/S 架构的科研管理系统的设计与实现[J].现代信息技术,2022,6(18):40-43.
- [13] 柯小龙,周春国.基于 MVVM 架构的解析木信息管理系统开发[J].森林工程,2021,37(01):18-27.
- [14] Nadeem Fakhar Malik, Aamer Nadeem, Muddassar Azam Sindhu. Achieving State Space Reduction in Generated Ajax Web Application State Machine[J], Intelligent Automation and Soft Computing, 2022, 33(1): 429-455.
- [15] 王伶俐,张传国.基于 NodeJS+Express 框架的轻应用定制平台的设计与实现[J].计算

- 机科学,2017,44(S2):596-599.
- [16] Saundariya K , Abirami M ,Senthil K R , et al. Webapp Service for Booking Handyman Using Mongodb, Express JS, React JS, Node JS[C]// 2021 3rd International Conference on Signal Processing and Communication (ICPSC). 2021.32(03):45-51.
- [17] 刘江涛,王亮亮,崔夏阳,吴庆茹.基于 Node.js 和 MongoDB 的铁路选线案例系统设计与实现[J].铁路计算机应用,2021,30(09):42-46.
- [18] 王志宇,郭士华.基于文档型非关系型数据库档案数据存储研究[J].档案学研究,2021,05:79-84.
- [19] 孔云,资芸,杨婷,郑磊,田春燕.基于 MongoDB 的数字资源访问日志存储模型构建[J].图书馆学刊,2022,44(09):78-86.
- [20] 张裔智,赵毅,汤小斌.MD5 算法研究[J].计算机科学,2008,07:295-297.
- [21] 李强,陈登峰.改进 MD5 加密算法在系统密码存储中的研究及应用[J].信息记录材料,2021,22(10):157-159.
- [22] 范展源,罗福强.JWT 认证技术及其在 WEB 中的应用[J].数字技术与应用,2016,02:114.
- [23] 高春庚.基于 Cookie 的会话跟踪技术及应用[J].信息记录材料,2021,22(10):23-24.
- [24] 周虎.一种基于 JWT 认证 token 刷新机制研究[J].软件工程,2019,22(12):18-20.
- [25] Mohammad Bajammal, Ali Mesbah. Web Canvas Testing Through Visual Inference[C], International Conference on Software Testing, Verification, and Validation, 2018,C2: 193-203.
- [26] 厉鹏,樊颖.数据结构中栈的应用[J].电脑学习,2008,01:61-62.
- [27] 周煜莹,崔岩松,王丹志,陈科良.基于自适应分片的大文件快速上传[J].计算机系统应用,2022,31(07):143-148.
- [28] Andrea Gallidabino, Cesare Pautasso. The LiquidWebWorker API for Horizontal Offloading of Stateless Computations[J], Journal of Web Engineering, 2019, 17(6-7): 405-448.
- [29] 于贾燕.社区互动对知识付费社区用户粘性的影响研究[D].山东大学,2021,8(05):35-40.
- [30] 李俊萌.计算机软件测试技术与开发应用策略分析[J].信息记录材料,2023,24(03):50-52.
- [31] 张清睿,黄松,孙乐乐.Web 功能自动化测试综述[J].软件导刊,2023,22(03):227-236.

致谢

时光飞逝，大学生活已然接近尾声。回望大学四年，虽说一事无成，却又收获良多，增长了我的见识，磨练了我的心智。感谢湖南工商大学为我提供了良好的学习环境，也因此让我遇到和看到了一群帮助我支持我的人。

首先感谢我的导师，赵珏老师。赵老师对待专业极其负责且认真，不管是在课堂上还是课后的比赛指导上，都展示出极强的专业性。同时她自律且努力，执行力超强。赵老师即是良师又是益友，为人亲切。不管是学习上、工作上还是生活上，她都非常愿意倾听学生的声音，为学生答疑解惑。本次毕业设计上，老师也给了我很多宝贵的建议，再次感谢赵老师。赵老师对待工作和生活的态度也给了我很大的启发，在以后的工作生活中，赵老师会一直是我追随学习的榜样。

非常感谢爱我家人，尤其是我的母亲。她教我温柔善良地对待世界，教我如何发光照亮别人。正因如此我才成为了现在的我，独立自主，遇到困难积极应对，面对未来绝不后退。

大学四年也认识了很多好朋友，他们大多有一种清澈的愚蠢，我想这可能是我们彼此结缘的原因吧。感谢他们会在我开心的时候陪我疯，在我难过的时候安慰开导我，我们相互学习，相互鼓励。初入大学校园的时候，我以为大学是一场孤独的试炼，直到后来我身边慢慢站满了人，感谢好朋友们让我的大学生活没那么枯燥。但是人生的确是一个人的旅行，不管是谁都是自己旅途的过客。大学即将毕业，旅行者会和朋友们说再见，踏上新的旅途，但是旅行者永远不会忘记旅途本身的意义，在欣赏沿途的风景时，他会怀念的。

最后我要感谢我自己，感谢自己依旧充满朝气，没有失去对世界的期待；感谢自己还相信这世界绝大部分人都是善良的，并且愿意以最大的善意对待别人。希望自己在以后的工作生活中也能不忘初心，永远乐观，永远快乐。

愿祖国繁荣昌盛，愿母校越办越好！